Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий

Кафедра вычислительной техники

ОТЧЁТ О ПРАКТИЧЕСКОЙ РАБОТЕ №4

по дисциплине
«Гибридные вычислительные системы»

«Умножение матриц с использованием тензорных ядер CUDA»

Преподаватель _____ С. А. Тарасов
подпись, дата / инициалы, фамилия

Студент КИ__-__Б,_____ _____ _._. Афонин
номер группы, зачетной книжкой / подпись, дата / инициалы, фамилия

Студент КИ__-__Б,_____ _____ _._. Никитин
номер группы, зачетной книжкой / подпись, дата / инициалы, фамилия

Красноярск 2026

СОДЕРЖАНИЕ

1 Цель и задание 3
2 Описание реализации 4
2.1 Структура проекта 4
2.2 Умножение матриц на тензорных ядрах 5
2.3 Кёрнел kernel_matmul_wmma 6
2.4 Загрузка тайлов и запись результата 8
2.5 Выбор геометрии блока 9
2.6 Паттерн «Стратегия» и оператор умножения 10
3 Тестирование 12
4 Методика вычислительного эксперимента 14
5 Результаты измерений 16
6 Анализ результатов 19
6.1 Вычислительная сложность 19
6.2 Ускорение 19
6.3 Предел производительности и узкое место 20
6.4 Точность 22
ЗАКЛЮЧЕНИЕ 24
ПРИЛОЖЕНИЕ А (обязательное) Исходный код программы 25

1 Цель и задание

Цель работы: получить навыки программирования тензорных ядер CUDA.

Задание:

– разработать кёрнел `kernel_matmul_wmma`, который принимает объекты `MatrixView` по значению и вычисляет произведение матриц, используя CUDA WMMA;

– перегрузить оператор `operator*` для класса `Matrix`, используя указанный кёрнел;

– используя Google Test, разработать модульные тесты для `operator*` для матриц A (m×k) и B (k×n), где m, n, k ∈ {16, 32, 64, 128, 256, 512}; эталон – результат аналогичной операции для `Eigen::Matrix<Eigen::half, Eigen::Dynamic, Eigen::Dynamic, Eigen::RowMajor>`, сравнение – методом `Eigen::MatrixXf::isApprox` с точностью 10^{-2};

– используя Google Benchmark и CUDA Events API, разработать бенчмарки для `operator*` для матриц n×n, n ∈ {16, 32, 64, 128, 256, 512, 1024}, без учёта времени выделения, копирования и освобождения памяти;

– построить графики реальной вычислительной сложности умножения матриц для новой и предыдущей версий `operator*`, а также график ускорения новой версии относительно предыдущей;

– объяснить экспериментальные результаты.

2 Описание реализации

2.1 Структура проекта

Проект является развитием практической работы №3 и собирается с помощью CMake и Ninja, используется стандарт C++20 и CUDA 13.3. Зависимости (Google Test 1.14, Google Benchmark 1.8.3, Eigen 3.4.0) подключаются через механизм FetchContent. Проект состоит из трёх подпроектов:

– `core` – библиотека с классами `Data`, `MatrixView`, `Matrix`, стратегиями умножения и кёрнелами;

– `tests` – модульные тесты на Google Test;

– `benchmarks` – бенчмарки на Google Benchmark.

В каталоге `scripts` находятся скрипты на Python (пакет plotly) для построения графиков, в каталоге `results` – результаты измерений. Полная реализация доступна в репозитории https://github.com/nameofthelove/gvs-afonin-nikitin (ветка work4). Ключевые фрагменты кода приведены в тексте отчёта, полный исходный код – в приложении А.

Классы `Data`, `MatrixView` и `Matrix` перенесены из работы №3 без изменений структуры: шаблоны инстанцированы для типа `half`, а алгоритм умножения по-прежнему задаётся стратегией. Новые и изменённые компоненты приведены в таблице 1.

Таблица 1 – Новые и изменённые компоненты библиотеки core

Компонент	Назначение
`kernel_matmul_wmma`	Умножение матриц half на тензорных ядрах (kernel.cuh)
`wmma_load_tile`	Совместная загрузка тайла из глобальной памяти в разделяемую
`launch_matmul_wmma`	Проверка геометрии блока на этапе компиляции, расчёт сетки, асинхронный запуск кёрнела
`WmmaMatmul<Y, X, K>`	Стратегия на основе kernel_matmul_wmma с размерами тайлов BlockItemsY, BlockItemsX, BlockItemsK
`default_matmul_strategy`	Стратегия по умолчанию: WmmaMatmul для half, SharedMemoryMatmul для остальных типов

2.2 Умножение матриц на тензорных ядрах

Тензорное ядро выполняет умножение небольших матриц с накоплением одной коллективной инструкцией варпа. Интерфейс WMMA (пространство имён `nvcuda::wmma`, заголовок `mma.h`) оперирует фрагментами – распределёнными по регистрам нитей варпа матрицами A, B и накопителем C. Операция `mma_sync` вычисляет C = C + A·B для фрагментов 16×16; фрагменты загружаются функцией `load_matrix_sync` и выгружаются функцией `store_matrix_sync`. Все эти функции вызываются совместно всеми 32 нитями варпа. Тензорные ядра работают только с малоразрядными типами, поэтому кёрнел написан для `half` (IEEE 754 binary16).

Схема алгоритма приведена на рисунке 1. Блок из 256 нитей (8 варпов) вычисляет тайл C размером BlockItemsY×BlockItemsX. Варпы образуют решётку 2×4, накопители находятся в регистрах (RF). Внутренняя размерность k проходится полосами ширины BlockItemsK: на каждом шаге нити блока загружают в разделяемую память тайл A (BlockItemsY×BlockItemsK) и тайл B (BlockItemsK×BlockItemsX), после чего каждый варп накапливает свою часть тайла C.

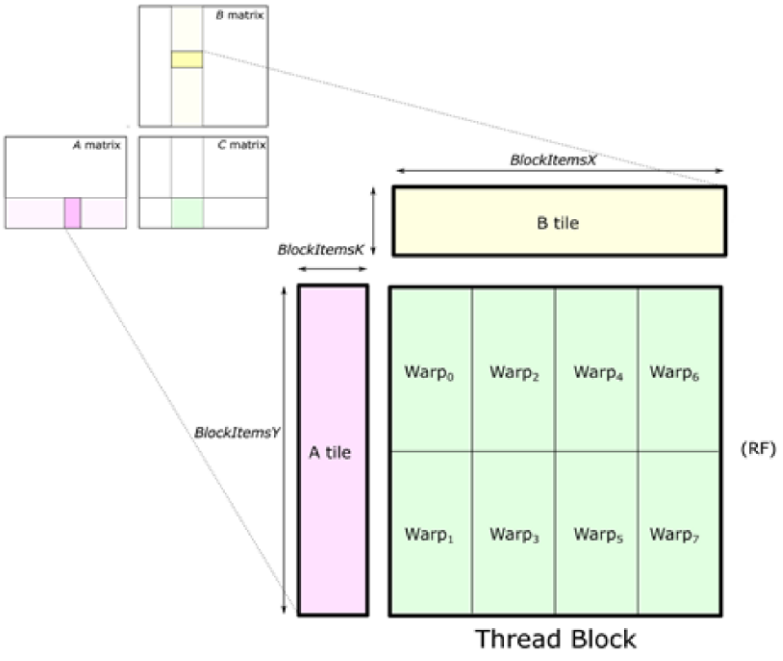


Рисунок 1 – Схема произведения матриц с помощью WMMA

Число арифметических операций при этом не меняется, зато один загруженный в тайл элемент используется многими операциями. Всего блоков $(n/X)\cdot(n/Y)$, полос n/K, на полосу загружается $K\cdot(X+Y)$ элементов по 2 байта, откуда объём чтения из глобальной памяти составляет $V = 2\cdot n^3\cdot(1/X + 1/Y)$ байт. Этот объём обратно пропорционален размеру тайла C – на этом основан выбор геометрии блока (раздел 2.5).

2.3 Кёрнел kernel_matmul_wmma

Реализация кёрнела приведена в листинге 1. Кёрнел – шаблон по размерам тайлов; блок ожидается одномерным (`blockDim = (256, 1)`), поэтому варп – это 32 нити с последовательными `threadIdx.x`.

Листинг 1 – Кёрнел kernel_matmul_wmma (core/include/kernel.cuh)

```
template <unsigned int BlockItemsY, unsigned int BlockItemsX, unsigned int
     BlockItemsK>
__global__ void kernel_matmul_wmma(MatrixView<half> a, MatrixView<half> b,
     MatrixView<half> c) {
    namespace wmma = nvcuda::wmma;


    constexpr unsigned int kWarpItemsY = BlockItemsY / kWmmaWarpRows;
    constexpr unsigned int kWarpItemsX = BlockItemsX / kWmmaWarpCols;
    constexpr unsigned int kFragsY = kWarpItemsY / kWmmaFragDim;
    constexpr unsigned int kFragsX = kWarpItemsX / kWmmaFragDim;


    using FragmentA = wmma::fragment<wmma::matrix_a, kWmmaFragDim, kWmmaFragDim,
     kWmmaFragDim,
                                     half, wmma::row_major>;
    using FragmentB = wmma::fragment<wmma::matrix_b, kWmmaFragDim, kWmmaFragDim,
     kWmmaFragDim,
                                     half, wmma::row_major>;
    using FragmentC = wmma::fragment<wmma::accumulator, kWmmaFragDim, kWmmaFragDim,
     kWmmaFragDim,
                                     half>;


    __shared__ alignas(16) half tile_a[BlockItemsY][BlockItemsK];
    __shared__ alignas(16) half tile_b[BlockItemsK][BlockItemsX];
    __shared__ alignas(16) half frag_c[kWmmaWarps][kWmmaFragDim][kWmmaFragDim];


    // Варпы занимают решётку по столбцам: Warp0 и Warp1 — первый столбец (рис. 1).
    const unsigned int warp = threadIdx.x / kWarpSize;
    const unsigned int warp_y = warp % kWmmaWarpRows * kWarpItemsY;
    const unsigned int warp_x = warp / kWmmaWarpRows * kWarpItemsX;
    const std::size_t block_row = static_cast<std::size_t>(blockIdx.y) * BlockItemsY;
    const std::size_t block_col = static_cast<std::size_t>(blockIdx.x) * BlockItemsX;


    FragmentC acc[kFragsY][kFragsX];
#pragma unroll
    for (unsigned int fy = 0; fy < kFragsY; ++fy) {
#pragma unroll
        for (unsigned int fx = 0; fx < kFragsX; ++fx) {
            wmma::fill_fragment(acc[fy][fx], half{});
```

```
        }
    }

    for (std::size_t strip = 0; strip < a.ncols(); strip += BlockItemsK) {
        wmma_load_tile<BlockItemsY, BlockItemsK>(&tile_a[0][0], a, block_row, strip);
        wmma_load_tile<BlockItemsK, BlockItemsX>(&tile_b[0][0], b, strip, block_col);
        __pipeline_commit();
        __pipeline_wait_prior(0);

        // Тайлы должны быть полностью загружены до начала вычислений.
        __syncthreads();

#pragma unroll
        for (unsigned int p = 0; p < BlockItemsK; p += kWmmaFragDim) {
            // Фрагмент A умножается на все фрагменты B варпа и наоборот, поэтому
     каждый
            // из них загружается в регистры один раз.
            FragmentA frag_a[kFragsY];
            FragmentB frag_b[kFragsX];
#pragma unroll
            for (unsigned int fy = 0; fy < kFragsY; ++fy) {
                wmma::load_matrix_sync(frag_a[fy], &tile_a[warp_y + fy *
     kWmmaFragDim][p],
                                       BlockItemsK);
            }
#pragma unroll
            for (unsigned int fx = 0; fx < kFragsX; ++fx) {
                wmma::load_matrix_sync(frag_b[fx], &tile_b[p][warp_x + fx *
     kWmmaFragDim],
                                       BlockItemsX);
            }
#pragma unroll
            for (unsigned int fy = 0; fy < kFragsY; ++fy) {
#pragma unroll
                for (unsigned int fx = 0; fx < kFragsX; ++fx) {
                    wmma::mma_sync(acc[fy][fx], frag_a[fy], frag_b[fx], acc[fy][fx]);
                }
            }
        }

        // Тайлы нельзя перезаписывать, пока все варпы блока не закончат их чтение.
        __syncthreads();
    }

    // Условие прямой записи одинаково для всех нитей варпа: зависит только от
     координат варпа.
    const bool stride_aligned = c.ncols() % (kWmmaStoreAlignment / sizeof(half)) == 0;
#pragma unroll
    for (unsigned int fy = 0; fy < kFragsY; ++fy) {
#pragma unroll
        for (unsigned int fx = 0; fx < kFragsX; ++fx) {
            const std::size_t row = block_row + warp_y + fy * kWmmaFragDim;
            const std::size_t col = block_col + warp_x + fx * kWmmaFragDim;
            half* frag_dst = &c(row, col);

            if (stride_aligned && row + kWmmaFragDim <= c.nrows() &&
                col + kWmmaFragDim <= c.ncols() &&
                reinterpret_cast<std::uintptr_t>(frag_dst) % kWmmaStoreAlignment == 0)
     {
                wmma::store_matrix_sync(frag_dst, acc[fy][fx],
                                        static_cast<unsigned int>(c.ncols()),
                                        wmma::mem_row_major);
                continue;
            }

            // Фрагмент выходит за границы C или строка C не выровнена: запись идёт
```

```
      через
            // разделяемую память. Варп читает только записанный им самим фрагмент,
      поэтому
            // барьер не требуется.
            half* buffer = &frag_c[warp][0][0];
            wmma::store_matrix_sync(buffer, acc[fy][fx], kWmmaFragDim,
      wmma::mem_row_major);
#pragma unroll
            for (unsigned int i = 0; i < kWmmaFragItems / kWarpSize; ++i) {
                const unsigned int item =
                    (threadIdx.x % kWarpSize) * (kWmmaFragItems / kWarpSize) + i;
                const std::size_t item_row = row + item / kWmmaFragDim;
                const std::size_t item_col = col + item % kWmmaFragDim;
                if (item_row < c.nrows() && item_col < c.ncols()) {
                    c(item_row, item_col) = buffer[item];
                }
            }
        }
    }
}
```

Ключевые решения:

– варпы занимают решётку 2×4 по столбцам, как на рисунке 1: $Warp_0$ и $Warp_1$ образуют первый столбец; при тайле больше 32×64 каждый варп вычисляет несколько фрагментов;

– фрагмент A умножается на все фрагменты B варпа (и наоборот), поэтому каждый фрагмент загружается в регистры один раз на полосу;

– первый `__syncthreads()` гарантирует, что тайлы загружены до начала вычислений, второй – что тайлы не будут перезаписаны, пока их читают другие варпы;

– циклы по фрагментам полностью разворачиваются (`#pragma unroll`), поэтому массивы фрагментов остаются в регистрах;

– условие прямой записи зависит только от координат варпа, поэтому все его нити выбирают одну ветвь и `store_matrix_sync` вызывается совместно.

2.4 Загрузка тайлов и запись результата

Загрузка тайлов оказалась главной статьёй расходов кёрнела (раздел 6.3), поэтому она выполняется пакетами (листинг 2). Каждая нить копирует свою часть тайла функцией `__pipeline_memcpy_async`, которая на Ampere читает до 16 байт одной инструкцией `LDG.128`. Если пакет выходит за границу матрицы или адрес не выровнен, нить переходит на поэлементное копирование с заменой отсутствующих элементов нулями. Благодаря этому кёрнел

обрабатывает размеры, не кратные 16, а также представления с произвольным начальным адресом.

Листинг 2 – Загрузка тайла (core/include/kernel.cuh)

```
template <unsigned int Rows, unsigned int Cols>
__device__ __forceinline__ void wmma_load_tile(half* tile, const MatrixView<half>&
     src,
                                                std::size_t row, std::size_t col) {
    constexpr unsigned int kThreadItems = Rows * Cols / kWmmaBlockThreads;
    constexpr unsigned int kPacketItems = kThreadItems < 8 ? kThreadItems : 8;
    constexpr unsigned int kPacketBytes = kPacketItems * sizeof(half);


#pragma unroll
    for (unsigned int packet = 0; packet < kThreadItems / kPacketItems; ++packet) {
        const unsigned int first = threadIdx.x * kThreadItems + packet * kPacketItems;
        const std::size_t src_row = row + first / Cols;
        const std::size_t src_col = col + first % Cols;
        const half* packet_src = &src(src_row, src_col);


        if (src_row < src.nrows() && src_col + kPacketItems <= src.ncols() &&
            reinterpret_cast<std::uintptr_t>(packet_src) % kPacketBytes == 0) {
            __pipeline_memcpy_async(&tile[first], packet_src, kPacketBytes);
        } else {
#pragma unroll
            for (unsigned int i = 0; i < kPacketItems; ++i) {
                const bool in_bounds = src_row < src.nrows() && src_col + i <
     src.ncols();
                tile[first + i] = in_bounds ? src(src_row, src_col + i) : half{};
            }
        }
    }
}
```

Фрагмент результата, лежащий в матрице C целиком и выровненный на 16 байт, записывается напрямую. Остальные фрагменты выгружаются в буфер `frag_c` разделяемой памяти и копируются поэлементно с проверкой границ: прямая запись фрагмента 16×16 вышла бы за пределы C.

2.5 Выбор геометрии блока

Первая версия кёрнела загружала тайлы поэлементно и записывала результат через промежуточный тайл C в разделяемой памяти. Предварительный замер показал, что при этом время слабо зависит от геометрии блока и составляет 706,1 мкс при n = 1024 для тайла 128×128×32. Переход на пакетную загрузку и прямую запись сократил его до 237,1 мкс, т. е. в 3,0 раза, что и указало на загрузку как на узкое место.

Далее выбрана геометрия блока (таблица 2). Значения n = 1024 приведены вместе с числом блоков и объёмом разделяемой памяти на блок.

Увеличение тайла C сокращает объём загрузки, но при тайле 256×256 блоков становится меньше, чем SM (при n = 512 их всего 4 на 20 SM), а при K = 64 блок занимает 36 КБ и на SM размещается не более трёх блоков вместо шести. Тайл 256×256×32 при n = 1024 быстрее на несколько процентов, но при n = 512 проигрывает вдвое. Поэтому по умолчанию выбран тайл 128×128×32: при n = 512 он занимает 16 блоками почти все SM. Геометрия 32×64×32 соответствует минимальному случаю рисунка 1: один фрагмент на варп.

Таблица 2 – Геометрии блока (n = 1024)

Y×X×K	Блоков	Разделяемая память, КБ	V, МБ	T, мкс	P, GFLOP/s
256×256×32	16	36	16,8	222,9	9633
128×256×32	32	28	25,2	278,7	7706
128×128×32	64	20	33,6	237,1	9056
128×128×16	64	12	33,6	249,8	8595
128×128×64	64	36	33,6	394,9	5438
64×128×32	128	16	50,3	304,7	7049
64×64×32	256	12	67,1	359,6	5971
32×64×16	512	7	100,7	472,4	4546
32×64×32	512	10	100,7	491,1	4373

2.6 Паттерн «Стратегия» и оператор умножения

Алгоритм подключён к иерархии стратегий из работы №3 (листинг 3). Тензорные ядра поддерживают только малоразрядные типы, поэтому `WmmaMatmul` реализует интерфейс `MatmulStrategy<half>`, а функция `default_matmul_strategy` выбирает реализацию на этапе компиляции. Сам `operator*` не изменился (листинг А.8): он проверяет размеры, создаёт матрицу результата, делегирует вычисление стратегии левого операнда и вызывает `cudaDeviceSynchronize`. Предыдущая версия остаётся доступной через `set_matmul_strategy`, что и используется при сравнении.

Листинг 3 – Стратегия WMMA (core/include/matmul_strategy.cuh)

```
// Умножение на тензорных ядрах (WMMA): только для half, размеры тайлов задают
     геометрию блока.
template <unsigned int BlockItemsY = kWmmaBlockItemsY, unsigned int BlockItemsX =
     kWmmaBlockItemsX,
          unsigned int BlockItemsK = kWmmaBlockItemsK>
class WmmaMatmul final : public MatmulStrategy<half> {
public:
```

```
    void multiply(MatrixView<half> a, MatrixView<half> b, MatrixView<half> c,
                  cudaStream_t stream = nullptr) const override {
        launch_matmul_wmma<BlockItemsY, BlockItemsX, BlockItemsK>(a, b, c, stream);
    }
};

// Стратегия, используемая Matrix::operator* по умолчанию: тензорные ядра поддерживают
     только
// half, остальные типы умножаются блочным алгоритмом с разделяемой памятью.
template <typename AtomT>
std::shared_ptr<const MatmulStrategy<AtomT>> default_matmul_strategy() {
    if constexpr (std::is_same_v<AtomT, half>) {
        static const auto strategy = std::make_shared<const WmmaMatmul<>>();
        return strategy;
    } else {
        static const auto strategy = std::make_shared<const
     SharedMemoryMatmul<AtomT>>();
        return strategy;
    }
}
```

3 Тестирование

Модульные тесты разработаны с использованием Google Test (листинг А.12). В каждом параметризованном тесте генерируются случайные матрицы A (m×k) и B (k×n) с элементами из [−1, 1], вычисляется эталонное произведение средствами Eigen на типе `Eigen::half` и произведение `operator*` на GPU. Формат `Eigen::half` совпадает с `half` побитово, а порядок хранения задан как RowMajor, поэтому обмен данными сводится к копированию памяти (листинг А.11). Результаты приводятся к `Eigen::MatrixXf` и сравниваются методом `isApprox` с точностью 10^{-2}. Наборы тестов приведены в таблице 3.

Таблица 3 – Наборы модульных тестов

Набор	Число тестов	Что проверяется
AllSizes/ MatmulParamTest	$6^3 = 216$	`operator*` со стратегией по умолчанию (`kernel_matmul_wmma`) для всех m, k, n из задания
AllStrategies/ MatmulStrategyParamTest	$5 \cdot 6^3 = 1080$	Те же размеры для стратегий `NaiveMatmul`, `SharedMemoryMatmul<16>` и геометрий 32×64×16, 64×128×64, 256×256×32
BoundarySizes/ MatmulUnalignedParamTest	$4 \cdot 7 \cdot 5 \cdot 7 = 980$	Размеры 1, 15, 17, 31, 33, 63, 129, не кратные фрагменту, для четырёх геометрий блока
MatmulTest, MatmulAccuracyTest, MatmulStrategyTest, MatrixTest, MatrixViewTest, DataTest	25	Граничные случаи, точность, свойства классов и стратегий, отсутствие утечек памяти

Метод `isApprox` проверяет условие $\|a - b\|_2 \le p \cdot \min(\|a\|_2, \|b\|_2)$, т. е. точность относительно нормы матрицы. Поэлементная абсолютная точность 10^{-2} для `half` недостижима: уже эталон Eigen отличается от точного произведения на 0,23 при n = 512 (раздел 6.4). В наборе BoundarySizes встречается результат из одного числа, близкого к нулю из-за сокращения слагаемых (m = 1, k = 17, n = 1): относительный критерий здесь определяется только округлением, хотя абсолютное отклонение от эталона составляет $5 \cdot 10^{-4}$. Поэтому для этого набора дополнительно допускается отклонение 10^{-2} по

норме; для размеров из задания применяется `isApprox` без изменений. Дополнительные тесты приведены в таблице 4.

Таблица 4 – Дополнительные тесты

Тест	Что проверяется
KnownSmallProduct	Произведение 2×3 · 3×2 с заранее известным результатом
IdentityIsNeutral	A·I = I·A = A для матрицы 129×129
VectorShapes	Строка 1×1000 на столбец 1000×1 и наоборот: много полос, вырожденная сетка
RepeatedProductIsStable	Повторное умножение даёт побитово тот же результат
UnalignedViews	Представления с начальным адресом, не выровненным на 16 байт: поэлементный путь загрузки и записи
DoesNotWriteOutsideResult	Запись не выходит за границу C: хвост буфера результата не изменяется
OperandsAreNotModified	Операнды не изменяются, результат в новом буфере
DimensionMismatchThrows	Несогласованные размеры – std::invalid_argument
ZeroInnerDimension GivesZeroMatrix	k = 0: результат m×n заполнен нулями
EmptyResult	m = 0: кёрнел не запускается, ошибок CUDA нет
FloatPrecision, DoublePrecision, IntegerProductIsExact	Для типов без поддержки WMMA `operator*` работает как раньше
NoDeviceMemoryLeak	После 50 умножений 512×512 с копированием объём свободной памяти GPU (`cudaMemGetInfo`) не изменился
WmmaIsMoreAccurate ThanSequentialHalf	Ошибка WMMA относительно точного произведения меньше ошибки Eigen::half
DefaultForHalfIsWmma, DefaultForFloatIsSharedMemory	Выбор стратегии по умолчанию зависит от типа элемента

Все 2301 тест выполнены успешно (Windows 11, MSVC 19.42, CUDA 13.3). Отсутствие утечек обеспечивается владением памятью через RAII-классы `Data` и `CudaEventTimer` и проверяется тестом `NoDeviceMemoryLeak`; все вызовы CUDA Runtime API проверяются функцией `check_cuda`.

4 Методика вычислительного эксперимента

Измерения выполнены на следующей конфигурации: GPU NVIDIA GeForce RTX 3050 Ti Laptop GPU (архитектура Ampere, compute capability 8.6, 20 SM, 4 ГБ GDDR6 с шиной 128 бит, кэш L2 2 МБ, разделяемая память до 48 КБ на блок), CPU Intel Core i7-12650H, ОС Windows 11, драйвер 616.92, CUDA 13.3, MSVC 19.42, сборка Release, архитектура sm_86.

Бенчмарки разработаны с использованием Google Benchmark (листинг А.15). Шаблонная функция `BM_Matrix_operator_mul` параметризуется стратегией:

– матрицы создаются и заполняются случайными значениями из [−1, 1] до цикла измерений, поэтому выделение, копирование и освобождение памяти в замер не входят;

– перед циклом выполняется прогревочный запуск;

– в цикле измеряется только время выполнения кёрнела – того же вызова `strategy.multiply`, который выполняет `operator*`, – с помощью пары CUDA Events (листинг А.14); время передаётся в Google Benchmark методом `SetIterationTime`;

– каждый бенчмарк повторяется 10 раз, в качестве результата используется медиана, на графиках также показано стандартное отклонение.

Предыдущая версия `operator*` измерена заново в тех же условиях в двух вариантах: на типе float, как в работе №3, и на типе half, т. е. на тех же данных, что и новая версия. Основным считается сравнение на half. Кроме размеров из задания измерены контрольные размеры n = 2048 и n = 4096: при $n \leq 512$ блоков меньше, чем SM, и асимптотику на них проверить нельзя.

Калибровочный бенчмарк `BM_Wmma_peak` измеряет предельный темп тензорных ядер: 120 блоков (все слоты 20 SM) выполняют `mma_sync` над фрагментами в регистрах без обращений к памяти. Частота GPU контролировалась утилитой nvidia-smi: `cudaDevAttrClockRate` сообщает

1485 МГц, тогда как под нагрузкой частота составляла 1845–1950 МГц и снижалась с ростом температуры (до 87 °C).

5 Результаты измерений

Результаты измерений приведены в таблице 5. Производительность вычислена исходя из $2n^3$ операций с плавающей точкой на одно умножение, ускорение – как S(n) = T_shmem(n) / T_wmma(n).

Таблица 5 – Время умножения матриц n×n (медиана, мкс), производительность (GFLOP/s) и ускорение

n	T shmem float	T shmem half	T wmma 128×128×32	T wmma 32×64×32	P shmem half	P wmma	S half	S float
16	13,2	15,3	17,9	21,8	0,5	0,5	0,86	0,74
32	15,5	19,1	18,2	19,0	3,4	3,6	1,05	0,85
64	17,7	18,4	17,1	18,7	28	31	1,08	1,04
128	15,4	16,9	18,9	17,1	248	222	0,90	0,81
256	54,1	54,1	26,0	19,1	620	1291	2,08	2,08
512	392,8	384,1	42,3	64,7	699	6347	9,08	9,29
1024	3280,8	3059,5	237,1	491,1	702	9056	12,90	13,84
2048	25769,3	25259,2	1588,4	3931,3	680	10816	15,90	16,22
4096	207837,0	204341,4	12741,7	31274,2	673	10787	16,04	16,31

Размеры 2048 и 4096 – контрольные, в задание не входят.

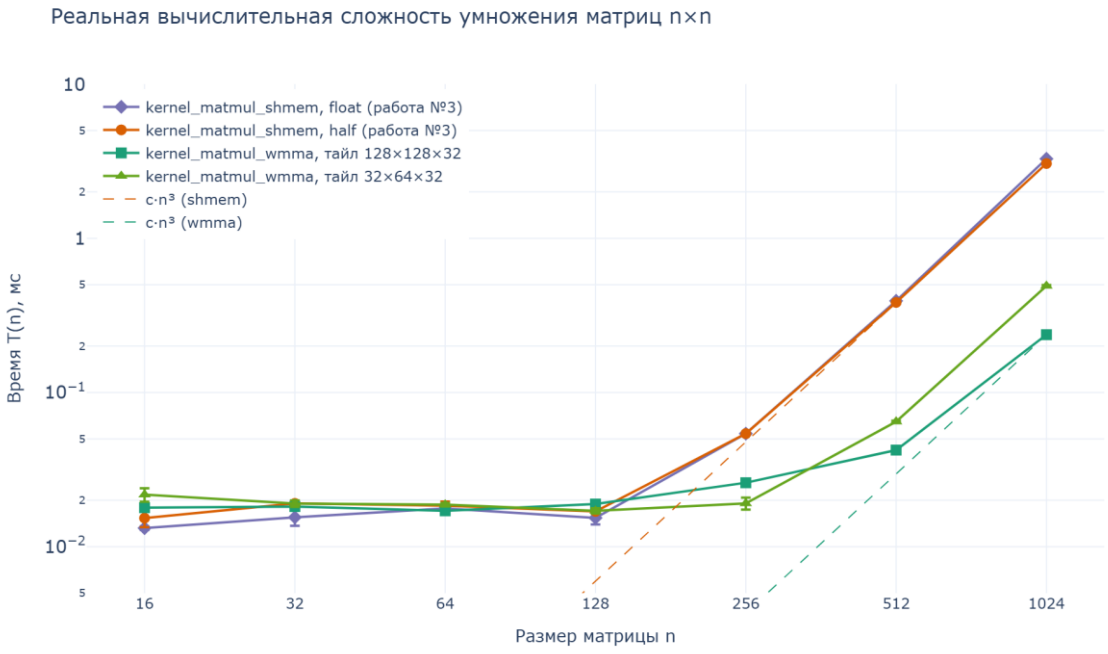


Рисунок 2 – Графики реальной вычислительной сложности T(n) (пунктир – теоретическая зависимость $c \cdot n^3$)

На рисунке 2 изображены графики реальной вычислительной сложности умножения матриц для предыдущей (`kernel_matmul_shmem`) и новой (`kernel_matmul_wmma`) реализаций `operator*` в логарифмическом масштабе. Пунктирные линии соответствуют зависимости $c \cdot n^3$, проведённой через последнюю точку серии.

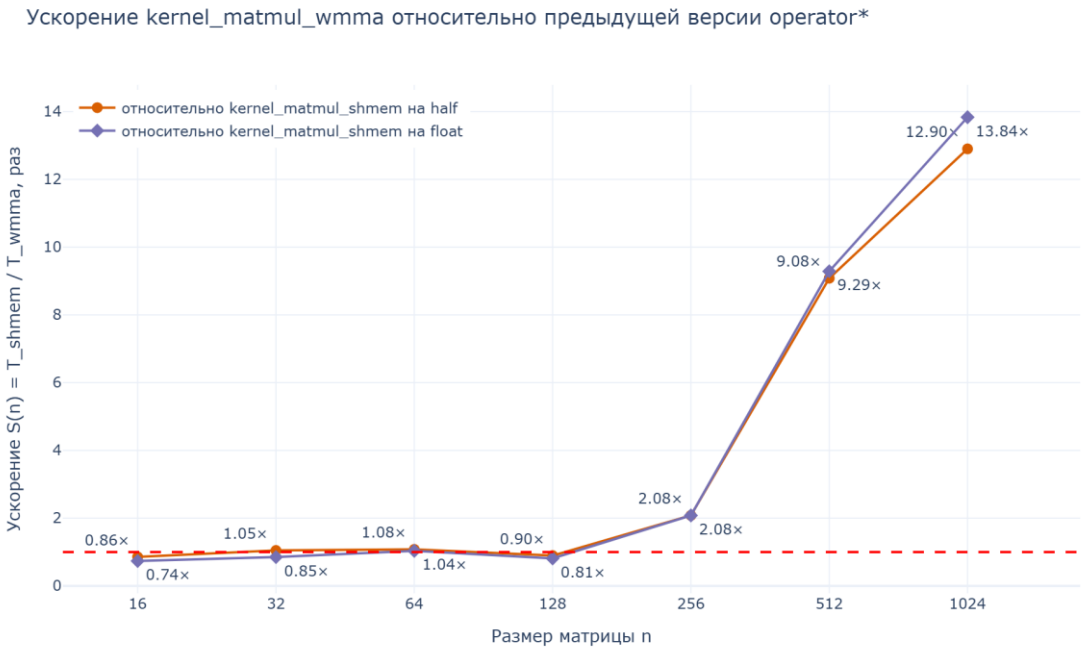


Рисунок 3 – График ускорения S(n) = T_shmem(n) / T_wmma(n)

На рисунке 3 изображён график ускорения новой реализации относительно предыдущей. Пунктирная линия S = 1 соответствует равной производительности.

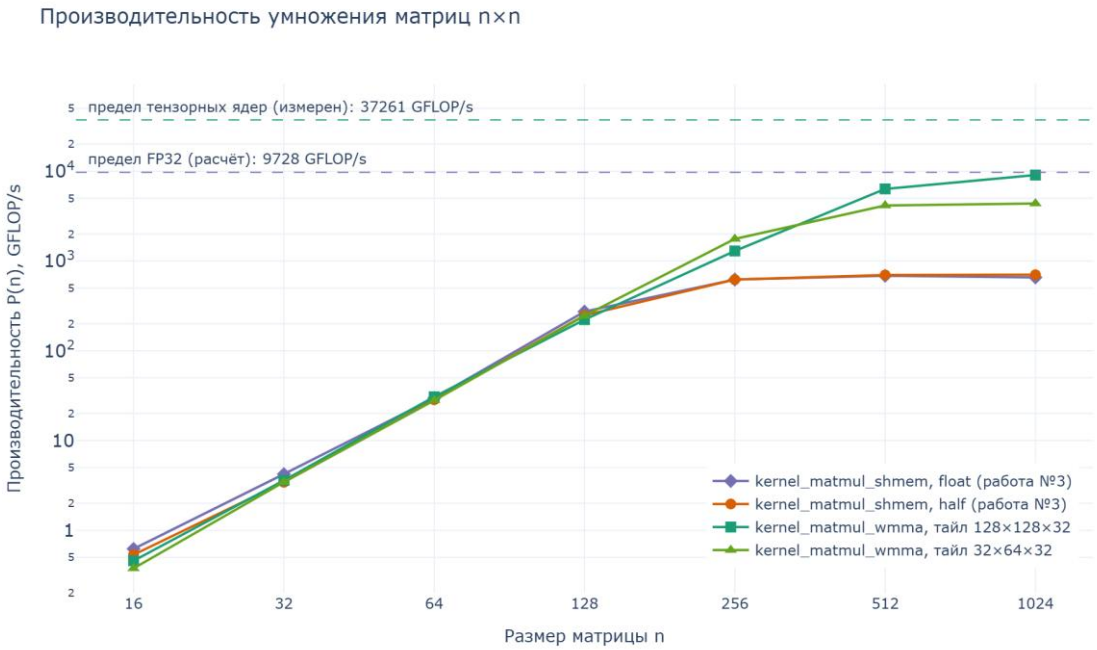


Рисунок 4 – Производительность P(n) и пределы вычислительных блоков GPU

На рисунке 4 изображена достигнутая производительность; пунктирные линии – измеренный предел тензорных ядер и расчётный предел FP32 (128 FMA/такт на SM).

6 Анализ результатов

6.1 Вычислительная сложность

Умножение матриц требует n^3 умножений и столько же сложений, т. е. имеет сложность $O(n^3)$. Тензорные ядра не изменяют число операций, а выполняют их группами, поэтому обе реализации должны различаться только константой. Показатель степени p в модели $T(n) = c \cdot n^p$ оценён методом наименьших квадратов (таблица 6).

Таблица 6 – Эмпирический показатель степени p в модели $T(n) = c \cdot n^p$

Реализация	p (16 ≤ n ≤ 1024)	p (256 ≤ n ≤ 1024)	p (n ≥ 1024)
shmem, float	1,24	2,96	2,99
shmem, half	1,18	2,91	3,03
wmma, 128×128×32	0,51	1,60	2,87
wmma, 32×64×32	0,61	2,34	3,00

Для предыдущей версии показатель при $n \geq 256$ близок к трём. Для `kernel_matmul_wmma` с тайлом 128×128 в диапазоне задания он заметно ниже (1,60), что не противоречит теории и объясняется двумя причинами. Во-первых, постоянные накладные расходы $t_0 \approx 18{,}7$ мкс (запуск кёрнела, запись событий в драйвере WDDM) сопоставимы со временем счёта вплоть до $n = 512$. Во-вторых, при $n \leq 512$ запускается всего 1, 4 и 16 блоков на 20 SM: все SM не заняты, и рост объёма работы частично поглощается ростом числа занятых SM. Поэтому чистое время $T - t_0$ при переходе от $n = 256$ к $n = 512$ растёт только в 3,2 раза вместо восьми.

Контрольные точки подтверждают асимптотику: при переходе от $n = 2048$ к $n = 4096$ время новой реализации растёт в 8,02 раза, предыдущей – в 8,09 раза при теоретическом значении 8. Таким образом, обе реализации имеют сложность $O(n^3)$ и различаются константой.

6.2 Ускорение

При $n \leq 128$ ускорения нет ($S = 0{,}86–1{,}08$): время всех реализаций определяется накладными расходами t_0, а при тайле 128×128 весь расчёт выполняется одним блоком на одном SM. Начиная с $n = 256$ новая реализация

быстрее: S = 2,08 при n = 256, 9,08 при n = 512 и 12,90 при n = 1024. На контрольных размерах ускорение выходит на насыщение: 15,90 при n = 2048 и 16,04 при n = 4096.

Ускорение значительно превышает отношение пиковых производительностей вычислительных блоков: предел тензорных ядер составляет 37261 GFLOP/s, расчётный предел FP32 – 9728 GFLOP/s, т. е. разница лишь в 3,8 раза. Причина в том, что предыдущая версия ограничена не вычислительными блоками, а чтением операндов. По дизассемблированному коду `cuobjdump -sass` внутренний цикл `kernel_matmul_shmem` содержит 16 инструкций FMA на 20 инструкций LDS, т. е. на каждую арифметическую инструкцию варпа приходится загрузка из разделяемой памяти, и производительность остаётся около 702 GFLOP/s, или 7 % пика FP32. На 16 инструкций варпа приходится 512 операций FMA, т. е. одна загрузка из разделяемой памяти на 26 операций. В `kernel_matmul_wmma` с тайлом 128×128×32 на 64 инструкции HMMA (65 536 операций FMA) приходится 24 инструкции LDSM, т. е. одна загрузка примерно на 2700 операций: данные тайла многократно используются из регистров.

Переход с float на half в предыдущей версии практически не меняет время (S на half и на float различаются менее чем на 10 % при $n \geq 256$): в коде 16 инструкций `FFMA` заменяются на 16 `HFMA2`, а число загрузок остаётся тем же – 20 инструкций LDS. Следовательно, разрядность сама по себе выигрыша не даёт, он обеспечивается именно тензорными ядрами.

6.3 Предел производительности и узкое место

Калибровочный бенчмарк показал предел 37261 GFLOP/s. По дизассемблированному коду один вызов `mma_sync` компилируется в 4 инструкции `HMMA.1688.F16`, каждая выполняет 16·8·8 = 1024 операции FMA. Пересчёт на частоту, замеренную одновременно с калибровкой (в разных прогонах 3290 мкс при 1950 МГц и 3429 мкс при 1845 МГц), даёт 502–509

FMA/такт на SM, т. е. примерно одну инструкцию HMMA на два такта на каждый SM.

Достигнутая производительность на контрольных размерах стабильна и составляет 10787 GFLOP/s, или 29 % от предела. Для объяснения разницы использована модель $T(n) = t_0 + 2n^3/P + V/B$, где P – измеренный предел, V – объём загрузки из раздела 2.2, B – эффективная пропускная способность тракта загрузки. Оценка B методом наименьших квадратов по геометриям, занимающим все SM и все слоты блоков (n = 1024), даёт $B \approx 240$ ГБ/с (рисунок 5). Параметр B включает все потери на пути загрузки, в том числе синхронизацию.

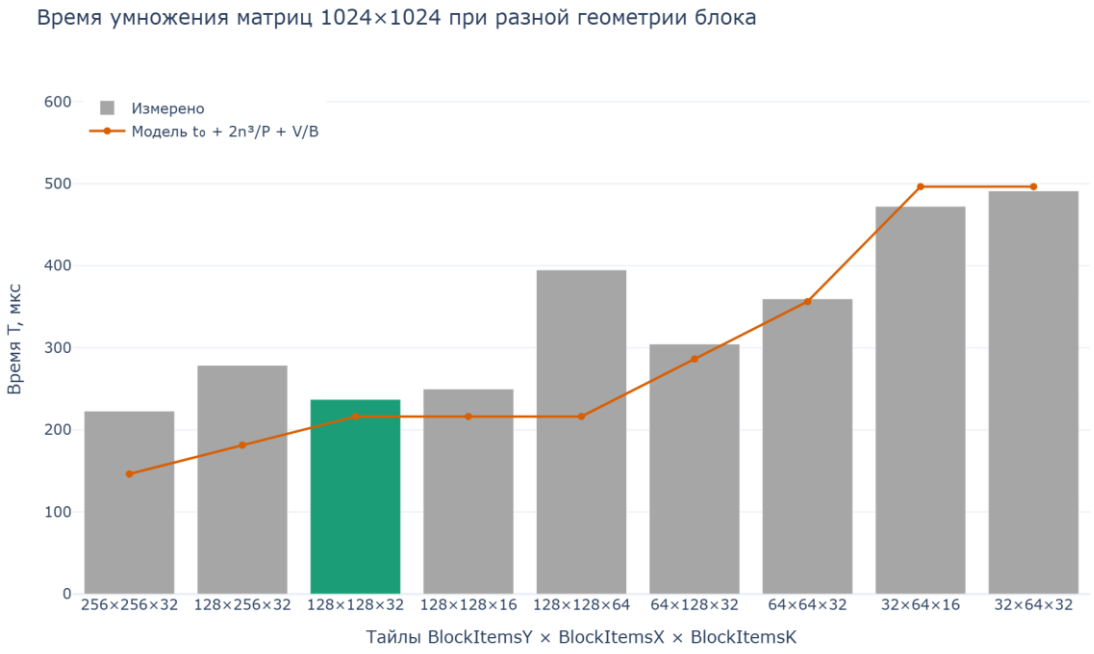


Рисунок 5 – Влияние геометрии блока при n = 1024 (линия – модель $t_0 + 2n^3/P + V/B$)

Модель описывает геометрии с полной загрузкой SM (128×128×16, 128×128×32, 64×128×32, 64×64×32, 32×64×16, 32×64×32) с отклонением не более 15 %. Геометрии выше модели – это как раз случаи неполной загрузки, отмеченные в разделе 2.5: 256×256×32 и 128×256×32 (мало блоков, много разделяемой памяти) и 128×128×64 (не более трёх блоков на SM). Независимая проверка: для тайла 128×128 на байт загрузки приходится 64 операции, и

модель предсказывает асимптотическую производительность 10864 GFLOP/s при измеренной 10787 GFLOP/s, т. е. расхождение не превышает 1 %.

Отсюда следует вывод: тензорные ядра используются примерно на треть, а около 71 % времени занимает загрузка тайлов. Значение B выше теоретической пропускной способности DRAM (6001 МГц·2·128 бит / 8 ≈ 192 ГБ/с), поскольку один и тот же тайл A читают все блоки строки сетки, и часть запросов обслуживается кэшем L2. Дальнейшего роста можно ожидать от двойной буферизации тайлов, при которой загрузка следующей полосы перекрывается с вычислениями над текущей (примитивы `cuda_pipeline.h` это позволяют), однако она удваивает объём разделяемой памяти.

6.4 Точность

Тензорные ядра дают ещё один, не очевидный заранее результат. В таблице 7 приведена погрешность относительно точного произведения, вычисленного в double (тест `DISABLED_AccuracyReport`, результаты в `results/accuracy.csv`).

Таблица 7 – Среднеквадратичная погрешность результата относительно точного произведения

n	Eigen::half	shmem, half	wmma	Eigen / wmma	Отн. погрешность wmma
16	0,00093	0,00074	0,00033	2,82	0,02 %
64	0,00336	0,00324	0,00118	2,85	0,04 %
256	0,01275	0,01261	0,00451	2,83	0,08 %
512	0,02506	0,02497	0,00885	2,83	0,12 %
1024	0,05018	0,05002	0,01773	2,83	0,17 %

Погрешность WMMA стабильно в 2,82–2,85 раза меньше, чем у последовательного накопления в half, и это отношение близко к $\sqrt{8} \approx 2{,}83$. Объяснение согласуется с форматом инструкции: `HMMA.1688` за один раз суммирует 8 произведений, поэтому результат округляется до half в 8 раз реже, а накопленная ошибка округления растёт как корень из числа округлений. Поэтому различие между результатом WMMA и эталоном Eigen определяется

в основном погрешностью самого эталона. Относительная погрешность растёт как √n, т. е. для больших матриц half следует применять с осторожностью.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы получены навыки программирования тензорных ядер CUDA.

Разработан кёрнел `kernel_matmul_wmma`, реализующий умножение матриц half с помощью CUDA WMMA по схеме из задания: тайлы A и B буферизуются в разделяемой памяти, блок из 8 варпов накапливает тайл C в регистрах. Кёрнел поддерживает размеры, не кратные 16, и невыровненные представления. `Matrix::operator*` использует его по умолчанию для типа half через паттерн «Стратегия». Корректность подтверждена 2301 модульным тестом с эталоном Eigen, включая все комбинации размеров из задания, граничные случаи и проверку отсутствия утечек памяти устройства.

Экспериментально подтверждена кубическая сложность: при удвоении n от 2048 до 4096 время растёт в 8,02 раза. Новая реализация быстрее предыдущей начиная с n = 256: ускорение на half составило 9,08 раза при n = 512, 12,90 раза при n = 1024 и насыщается на уровне около 16 раз. При $n \leq 128$ ускорения нет из-за постоянных накладных расходов на запуск. Достигнута производительность 10787 GFLOP/s при измеренном пределе тензорных ядер 37261 GFLOP/s; разница объясняется загрузкой тайлов, занимающей около 71 % времени, что подтверждено моделью: её отклонение от измерений не превышает 15 % для геометрий с полной загрузкой SM. Дополнительно установлено, что WMMA примерно в 2,8 раза точнее последовательного накопления в half.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код программы

Листинг А.1 – Файл `CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.24)

if(POLICY CMP0135)
  cmake_policy(SET CMP0135 NEW)
endif()

project(work4 LANGUAGES CXX CUDA)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CUDA_STANDARD 20)
set(CMAKE_CUDA_STANDARD_REQUIRED ON)

if(NOT CMAKE_BUILD_TYPE AND NOT CMAKE_CONFIGURATION_TYPES)
  set(CMAKE_BUILD_TYPE Release CACHE STRING "Build type" FORCE)
endif()

# Архитектура GPU (Ampere: RTX 3050 Ti -> sm_86); переопределяется через -
     DCMAKE_CUDA_ARCHITECTURES
if(NOT DEFINED CMAKE_CUDA_ARCHITECTURES)
  set(CMAKE_CUDA_ARCHITECTURES 86)
endif()

if(MSVC)
  # Исходники в UTF-8 (комментарии на русском); C4068 — прагмы nvcc в заголовках Eigen
  add_compile_options(
    $<$<COMPILE_LANGUAGE:CXX>:/utf-8>
    $<$<COMPILE_LANGUAGE:CUDA>:-Xcompiler=/utf-8,/wd4068>
  )
endif()

find_package(CUDAToolkit REQUIRED)

include(FetchContent)

# Google Test
FetchContent_Declare(
  googletest
  URL https://github.com/google/googletest/archive/refs/tags/v1.14.0.zip
)
set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
set(INSTALL_GTEST OFF CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(googletest)

# Google Benchmark
FetchContent_Declare(
  googlebenchmark
  URL https://github.com/google/benchmark/archive/refs/tags/v1.8.3.zip
)
set(BENCHMARK_ENABLE_TESTING OFF CACHE BOOL "" FORCE)
set(BENCHMARK_ENABLE_INSTALL OFF CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(googlebenchmark)
```

```
# Eigen 3
FetchContent_Declare(
  eigen
  GIT_REPOSITORY https://gitlab.com/libeigen/eigen.git
  GIT_TAG 3.4.0
  GIT_SHALLOW TRUE
)
set(EIGEN_BUILD_DOC OFF CACHE BOOL "" FORCE)
set(EIGEN_BUILD_TESTING OFF CACHE BOOL "" FORCE)
set(EIGEN_BUILD_PKGCONFIG OFF CACHE BOOL "" FORCE)
set(BUILD_TESTING OFF CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(eigen)


enable_testing()


add_subdirectory(core)
add_subdirectory(tests)
add_subdirectory(benchmarks)
```

Листинг А.2 – Файл `core/CMakeLists.txt`

```
add_library(core STATIC
    src/matrix.cu
)


target_include_directories(core PUBLIC
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)


target_link_libraries(core PUBLIC
    CUDA::cudart
)
```

Листинг А.3 – Файл `core/include/cuda_check.cuh`

```
#pragma once


#include <cuda_runtime.h>
#include <stdexcept>
#include <string>


// Преобразует код ошибки CUDA Runtime API в исключение std::runtime_error.
inline void check_cuda(cudaError_t err, const char* what) {
    if (err != cudaSuccess) {
        throw std::runtime_error(std::string(what) + ": " + cudaGetErrorString(err));
    }
}
```

Листинг А.4 – Файл `core/include/data.cuh`

```
#pragma once


#include <cstddef>
#include <cuda_runtime.h>
#include <utility>


#include "cuda_check.cuh"


// Владеющий RAII-буфер в глобальной памяти устройства.
template <typename AtomT>
```

```
class Data {
private:
    std::size_t size_;
    AtomT* data_;


public:
    explicit Data(std::size_t size) : size_(size), data_(nullptr) {
        if (size_ > 0) {
            check_cuda(cudaMalloc(&data_, size_ * sizeof(AtomT)), "cudaMalloc");
        }
    }


    ~Data() {
        if (data_ != nullptr) {
            cudaFree(data_);
        }
    }


    Data(const Data& other) : Data(other.size_) { //cop
        if (size_ > 0) {
            // При исключении память освободит деструктор делегирующего конструктора.
            check_cuda(cudaMemcpy(data_, other.data_, size_ * sizeof(AtomT),
                                  cudaMemcpyDeviceToDevice),
                       "cudaMemcpy (D2D)");
        }
    }


    Data(Data&& other) noexcept
        : size_(std::exchange(other.size_, 0)), data_(std::exchange(other.data_,
     nullptr)) {}


    Data& operator=(const Data& other) {
        if (this != &other) {
            Data temp(other);
            std::swap(size_, temp.size_);
            std::swap(data_, temp.data_);
        }
        return *this;
    }


    Data& operator=(Data&& other) noexcept {
        if (this != &other) {
            if (data_ != nullptr) {
                cudaFree(data_);
            }
            size_ = std::exchange(other.size_, 0);
            data_ = std::exchange(other.data_, nullptr);
        }
        return *this;
    }


    void copy_to_host(AtomT* host_ptr) const {
        if (size_ == 0) {
            return;
        }
        check_cuda(cudaMemcpy(host_ptr, data_, size_ * sizeof(AtomT),
     cudaMemcpyDeviceToHost),
                   "cudaMemcpy (D2H)");
    }


    void copy_from_host(const AtomT* host_ptr) {
        if (size_ == 0) {
            return;
        }
        check_cuda(cudaMemcpy(data_, host_ptr, size_ * sizeof(AtomT),
```

```
     cudaMemcpyHostToDevice),
                  "cudaMemcpy (H2D)");
    }

    AtomT* data() { return data_; }
    const AtomT* data() const { return data_; }
    std::size_t size() const { return size_; }
};
```

Листинг А.5 – Файл `core/include/matrix_view.cuh`

```
#pragma once

#include <cstddef>
#include <cuda_runtime.h>
#include <type_traits>

// Невладеющее представление матрицы nrows x ncols в линейной памяти (row-major).
// Тривиально копируемо, поэтому передаётся в кёрнелы по значению.
template <typename AtomT>
class MatrixView {
private:
    AtomT* data_;
    std::size_t nrows_;
    std::size_t ncols_;

public:
    __host__ __device__ MatrixView(AtomT* data, std::size_t nrows, std::size_t ncols)
        : data_(data), nrows_(nrows), ncols_(ncols) {}

    __host__ __device__ std::size_t size() const { return nrows_ * ncols_; }
    __host__ __device__ std::size_t nrows() const { return nrows_; }
    __host__ __device__ std::size_t ncols() const { return ncols_; }

    // Доступ по линейному индексу.
    __host__ __device__ AtomT& operator[](std::size_t n) { return data_[n]; }
    __host__ __device__ const AtomT& operator[](std::size_t n) const { return
     data_[n]; }

    // Доступ по (строка, столбец): элемент (i, j) лежит по смещению i * ncols + j.
    __host__ __device__ AtomT& operator()(std::size_t i, std::size_t j) {
        return data_[i * ncols_ + j];
    }
    __host__ __device__ const AtomT& operator()(std::size_t i, std::size_t j) const {
        return data_[i * ncols_ + j];
    }
};

static_assert(std::is_trivially_copyable_v<MatrixView<float>>,
              "MatrixView must be trivially copyable to be passed to kernels by
     value");
```

Листинг А.6 – Файл `core/include/kernel.cuh`

```
#pragma once

#include <cstddef>
#include <cstdint>
#include <cuda_fp16.h>
#include <cuda_pipeline.h>
#include <cuda_runtime.h>
#include <mma.h>
```

```
#include "cuda_check.cuh"
#include "matrix_view.cuh"


// Сторона квадратного блока нитей для наивного кёрнела (16 x 16 = 256 нитей в блоке).
inline constexpr unsigned int kMatmulBlockDim = 16;


// Сторона квадратного тайла для кёрнела с разделяемой памятью по умолчанию.
// 16 x 16 = 256 нитей: на SM архитектуры Ampere (sm_86, до 1536 нитей) помещается 6
     блоков,
// тогда как при тайле 32 x 32 (1024 нити) — только один, что снижает загрузку SM до
     67 %.
inline constexpr unsigned int kMatmulTileDim = 16;


// Геометрия тензорного кёрнела: блок из 8 варпов образует решётку 2 x 4 (рис. 1
     задания),
// каждый варп вычисляет в регистрах прямоугольник C из фрагментов 16 x 16.
inline constexpr unsigned int kWmmaFragDim = 16;
inline constexpr unsigned int kWmmaFragItems = kWmmaFragDim * kWmmaFragDim;
inline constexpr unsigned int kWarpSize = 32;
inline constexpr unsigned int kWmmaWarpRows = 2;
inline constexpr unsigned int kWmmaWarpCols = 4;
inline constexpr unsigned int kWmmaWarps = kWmmaWarpRows * kWmmaWarpCols;
inline constexpr unsigned int kWmmaBlockThreads = kWmmaWarps * kWarpSize;


// store_matrix_sync требует выравнивания каждой строки фрагмента на 16 байт.
inline constexpr unsigned int kWmmaStoreAlignment = 16;


// Размеры тайлов по умолчанию: BlockItemsY x BlockItemsX — тайл C, BlockItemsK —
     ширина
// полосы по внутренней размерности. Объём загрузки тайлов на одну операцию MMA
     обратно
// пропорционален размеру тайла C, поэтому тайл C выбран большим: каждый варп
     вычисляет 8
// фрагментов. Тайл 256 x 256 при n <= 512 не загружает все SM (не более 4 блоков).
inline constexpr unsigned int kWmmaBlockItemsY = 128;
inline constexpr unsigned int kWmmaBlockItemsX = 128;
inline constexpr unsigned int kWmmaBlockItemsK = 32;


// Наивное умножение C = A * B без разделяемой памяти (версия из практической работы
     №2).
// Двумерная сетка: ось x — столбцы C, ось y — строки C; одна нить вычисляет один
     элемент C.
template <typename AtomT>
__global__ void kernel_matmul_naive(MatrixView<AtomT> a, MatrixView<AtomT> b,
                                    MatrixView<AtomT> c) {
    const std::size_t row = static_cast<std::size_t>(blockIdx.y) * blockDim.y +
     threadIdx.y;
    const std::size_t col = static_cast<std::size_t>(blockIdx.x) * blockDim.x +
     threadIdx.x;


    if (row < c.nrows() && col < c.ncols()) {
        AtomT acc{};
        for (std::size_t p = 0; p < a.ncols(); ++p) {
            acc += a(row, p) * b(p, col);
        }
        c(row, col) = acc;
    }
}


// Блочное умножение C = A * B с разделяемой памятью.
// Блок нитей TileDim x TileDim вычисляет тайл C того же размера. Внутренняя
     размерность k
// проходится полосами ширины TileDim: на каждом шаге нити блока совместно загружают
     тайл A
```

```
// и тайл B в разделяемую память (каждая нить — по одному элементу каждого тайла),
     после чего
// каждая нить накапливает скалярное произведение строки тайла A на столбец тайла B.
// Элементы за границами матриц заменяются нулями, поэтому размеры могут быть не
     кратны TileDim.
// Кёрнел ожидает blockDim = (TileDim, TileDim).
template <typename AtomT, unsigned int TileDim>
__global__ void kernel_matmul_shmem(MatrixView<AtomT> a, MatrixView<AtomT> b,
                                    MatrixView<AtomT> c) {
    __shared__ AtomT tile_a[TileDim][TileDim];
    __shared__ AtomT tile_b[TileDim][TileDim];

    const unsigned int tx = threadIdx.x;
    const unsigned int ty = threadIdx.y;
    const std::size_t row = static_cast<std::size_t>(blockIdx.y) * TileDim + ty;
    const std::size_t col = static_cast<std::size_t>(blockIdx.x) * TileDim + tx;

    AtomT acc{};
    for (std::size_t tile_start = 0; tile_start < a.ncols(); tile_start += TileDim) {
        // Соседние по tx нити читают соседние элементы строки — доступ объединяется.
        const std::size_t a_col = tile_start + tx;
        const std::size_t b_row = tile_start + ty;
        tile_a[ty][tx] = (row < a.nrows() && a_col < a.ncols()) ? a(row, a_col) :
     AtomT{};
        tile_b[ty][tx] = (b_row < b.nrows() && col < b.ncols()) ? b(b_row, col) :
     AtomT{};

        // Тайлы должны быть полностью загружены до начала вычислений.
        __syncthreads();

#pragma unroll
        for (unsigned int p = 0; p < TileDim; ++p) {
            // tile_a[ty][p] одинаков для всего варпа (broadcast),
            // tile_b[p][tx] — последовательные слова, конфликтов банков нет.
            acc += tile_a[ty][p] * tile_b[p][tx];
        }

        // Тайлы нельзя перезаписывать, пока все нити блока не закончат их чтение.
        __syncthreads();
    }

    // Проверка выполняется после цикла: все нити блока, включая лежащие за границами
     C,
    // должны участвовать в загрузке тайлов и доходить до __syncthreads.
    if (row < c.nrows() && col < c.ncols()) {
        c(row, col) = acc;
    }
}

// Нити блока совместно копируют прямоугольник Rows x Cols матрицы src с началом в
     (row, col)
// в тайл разделяемой памяти. Каждая нить копирует Rows * Cols / kWmmaBlockThreads
// последовательных элементов пакетами по kPacketItems: если пакет лежит в матрице и
     выровнен,
// он читается одной векторной инструкцией, иначе — поэлементно с заменой нулями
     элементов
// за границами src.
template <unsigned int Rows, unsigned int Cols>
__device__ __forceinline__ void wmma_load_tile(half* tile, const MatrixView<half>&
     src,
                                               std::size_t row, std::size_t col) {
    constexpr unsigned int kThreadItems = Rows * Cols / kWmmaBlockThreads;
    constexpr unsigned int kPacketItems = kThreadItems < 8 ? kThreadItems : 8;
    constexpr unsigned int kPacketBytes = kPacketItems * sizeof(half);
```

```
#pragma unroll
    for (unsigned int packet = 0; packet < kThreadItems / kPacketItems; ++packet) {
        const unsigned int first = threadIdx.x * kThreadItems + packet * kPacketItems;
        const std::size_t src_row = row + first / Cols;
        const std::size_t src_col = col + first % Cols;
        const half* packet_src = &src(src_row, src_col);


        if (src_row < src.nrows() && src_col + kPacketItems <= src.ncols() &&
            reinterpret_cast<std::uintptr_t>(packet_src) % kPacketBytes == 0) {
            __pipeline_memcpy_async(&tile[first], packet_src, kPacketBytes);
        } else {
#pragma unroll
            for (unsigned int i = 0; i < kPacketItems; ++i) {
                const bool in_bounds = src_row < src.nrows() && src_col + i <
     src.ncols();
                tile[first + i] = in_bounds ? src(src_row, src_col + i) : half{};
            }
        }
    }
}


// Умножение C = A * B на тензорных ядрах (CUDA WMMA, half).
// Блок из 8 варпов вычисляет тайл C размером BlockItemsY x BlockItemsX. Внутренняя
     размерность k
// проходится полосами ширины BlockItemsK: на каждом шаге нити блока совместно
     загружают в
// разделяемую память тайл A (BlockItemsY x BlockItemsK) и тайл B (BlockItemsK x
     BlockItemsX),
// после чего каждый варп накапливает в регистрах свою часть тайла C операциями
     mma_sync.
// Фрагмент, лежащий в матрице C целиком, записывается в неё напрямую, остальные —
     через буфер
// в разделяемой памяти с проверкой границ, поэтому размеры могут быть не кратны 16.
// Кёрнел ожидает blockDim = (kWmmaBlockThreads, 1).
template <unsigned int BlockItemsY, unsigned int BlockItemsX, unsigned int
     BlockItemsK>
__global__ void kernel_matmul_wmma(MatrixView<half> a, MatrixView<half> b,
     MatrixView<half> c) {
    namespace wmma = nvcuda::wmma;


    constexpr unsigned int kWarpItemsY = BlockItemsY / kWmmaWarpRows;
    constexpr unsigned int kWarpItemsX = BlockItemsX / kWmmaWarpCols;
    constexpr unsigned int kFragsY = kWarpItemsY / kWmmaFragDim;
    constexpr unsigned int kFragsX = kWarpItemsX / kWmmaFragDim;


    using FragmentA = wmma::fragment<wmma::matrix_a, kWmmaFragDim, kWmmaFragDim,
     kWmmaFragDim,
                                     half, wmma::row_major>;
    using FragmentB = wmma::fragment<wmma::matrix_b, kWmmaFragDim, kWmmaFragDim,
     kWmmaFragDim,
                                     half, wmma::row_major>;
    using FragmentC = wmma::fragment<wmma::accumulator, kWmmaFragDim, kWmmaFragDim,
     kWmmaFragDim,
                                     half>;


    __shared__ alignas(16) half tile_a[BlockItemsY][BlockItemsK];
    __shared__ alignas(16) half tile_b[BlockItemsK][BlockItemsX];
    __shared__ alignas(16) half frag_c[kWmmaWarps][kWmmaFragDim][kWmmaFragDim];


    // Варпы занимают решётку по столбцам: Warp0 и Warp1 — первый столбец (рис. 1).
    const unsigned int warp = threadIdx.x / kWarpSize;
    const unsigned int warp_y = warp % kWmmaWarpRows * kWarpItemsY;
    const unsigned int warp_x = warp / kWmmaWarpRows * kWarpItemsX;
    const std::size_t block_row = static_cast<std::size_t>(blockIdx.y) * BlockItemsY;
    const std::size_t block_col = static_cast<std::size_t>(blockIdx.x) * BlockItemsX;
```

```
    FragmentC acc[kFragsY][kFragsX];
#pragma unroll
    for (unsigned int fy = 0; fy < kFragsY; ++fy) {
#pragma unroll
        for (unsigned int fx = 0; fx < kFragsX; ++fx) {
            wmma::fill_fragment(acc[fy][fx], half{});
        }
    }


    for (std::size_t strip = 0; strip < a.ncols(); strip += BlockItemsK) {
        wmma_load_tile<BlockItemsY, BlockItemsK>(&tile_a[0][0], a, block_row, strip);
        wmma_load_tile<BlockItemsK, BlockItemsX>(&tile_b[0][0], b, strip, block_col);
        __pipeline_commit();
        __pipeline_wait_prior(0);


        // Тайлы должны быть полностью загружены до начала вычислений.
        __syncthreads();


#pragma unroll
        for (unsigned int p = 0; p < BlockItemsK; p += kWmmaFragDim) {
            // Фрагмент A умножается на все фрагменты B варпа и наоборот, поэтому
     каждый
            // из них загружается в регистры один раз.
            FragmentA frag_a[kFragsY];
            FragmentB frag_b[kFragsX];
#pragma unroll
            for (unsigned int fy = 0; fy < kFragsY; ++fy) {
                wmma::load_matrix_sync(frag_a[fy], &tile_a[warp_y + fy *
     kWmmaFragDim][p],
                                       BlockItemsK);
            }
#pragma unroll
            for (unsigned int fx = 0; fx < kFragsX; ++fx) {
                wmma::load_matrix_sync(frag_b[fx], &tile_b[p][warp_x + fx *
     kWmmaFragDim],
                                       BlockItemsX);
            }
#pragma unroll
            for (unsigned int fy = 0; fy < kFragsY; ++fy) {
#pragma unroll
                for (unsigned int fx = 0; fx < kFragsX; ++fx) {
                    wmma::mma_sync(acc[fy][fx], frag_a[fy], frag_b[fx], acc[fy][fx]);
                }
            }
        }


        // Тайлы нельзя перезаписывать, пока все варпы блока не закончат их чтение.
        __syncthreads();
    }


    // Условие прямой записи одинаково для всех нитей варпа: зависит только от
     координат варпа.
    const bool stride_aligned = c.ncols() % (kWmmaStoreAlignment / sizeof(half)) == 0;
#pragma unroll
    for (unsigned int fy = 0; fy < kFragsY; ++fy) {
#pragma unroll
        for (unsigned int fx = 0; fx < kFragsX; ++fx) {
            const std::size_t row = block_row + warp_y + fy * kWmmaFragDim;
            const std::size_t col = block_col + warp_x + fx * kWmmaFragDim;
            half* frag_dst = &c(row, col);


            if (stride_aligned && row + kWmmaFragDim <= c.nrows() &&
                col + kWmmaFragDim <= c.ncols() &&
                reinterpret_cast<std::uintptr_t>(frag_dst) % kWmmaStoreAlignment == 0)
     {
                wmma::store_matrix_sync(frag_dst, acc[fy][fx],
```

```
                                        static_cast<unsigned int>(c.ncols()),
                                        wmma::mem_row_major);
                continue;
            }

            // Фрагмент выходит за границы C или строка C не выровнена: запись идёт
     через
            // разделяемую память. Варп читает только записанный им самим фрагмент,
     поэтому
            // барьер не требуется.
            half* buffer = &frag_c[warp][0][0];
            wmma::store_matrix_sync(buffer, acc[fy][fx], kWmmaFragDim,
     wmma::mem_row_major);
#pragma unroll
            for (unsigned int i = 0; i < kWmmaFragItems / kWarpSize; ++i) {
                const unsigned int item =
                    (threadIdx.x % kWarpSize) * (kWmmaFragItems / kWarpSize) + i;
                const std::size_t item_row = row + item / kWmmaFragDim;
                const std::size_t item_col = col + item % kWmmaFragDim;
                if (item_row < c.nrows() && item_col < c.ncols()) {
                    c(item_row, item_col) = buffer[item];
                }
            }
        }
    }
}


constexpr bool is_power_of_two(unsigned int value) {
    return value != 0 && (value & (value - 1)) == 0;
}


// Размер сетки, покрывающей матрицу с прямоугольниками block_y x block_x.
template <typename AtomT>
dim3 matmul_grid(const MatrixView<AtomT>& c, unsigned int block_y, unsigned int
     block_x) {
    return dim3(static_cast<unsigned int>((c.ncols() + block_x - 1) / block_x),
                static_cast<unsigned int>((c.nrows() + block_y - 1) / block_y));
}


// Асинхронный запуск kernel_matmul_naive; синхронизацию выполняет вызывающая сторона.
template <typename AtomT>
void launch_matmul_naive(MatrixView<AtomT> a, MatrixView<AtomT> b, MatrixView<AtomT>
     c,
                         cudaStream_t stream = nullptr) {
    if (c.size() == 0) {
        return;  // Пустая сетка — недопустимая конфигурация запуска.
    }

    const dim3 block(kMatmulBlockDim, kMatmulBlockDim);
    const dim3 grid = matmul_grid(c, kMatmulBlockDim, kMatmulBlockDim);
    kernel_matmul_naive<<<grid, block, 0, stream>>>(a, b, c);
    check_cuda(cudaGetLastError(), "kernel_matmul_naive launch");
}


// Асинхронный запуск kernel_matmul_shmem; синхронизацию выполняет вызывающая сторона.
template <typename AtomT, unsigned int TileDim = kMatmulTileDim>
void launch_matmul_shmem(MatrixView<AtomT> a, MatrixView<AtomT> b, MatrixView<AtomT>
     c,
                         cudaStream_t stream = nullptr) {
    static_assert(TileDim > 0 && TileDim * TileDim <= 1024,
                  "Block of TileDim x TileDim threads exceeds the CUDA limit of
     1024");

    if (c.size() == 0) {
        return;  // Пустая сетка — недопустимая конфигурация запуска.
    }
```

```
    const dim3 block(TileDim, TileDim);
    const dim3 grid = matmul_grid(c, TileDim, TileDim);
    kernel_matmul_shmem<AtomT, TileDim><<<grid, block, 0, stream>>>(a, b, c);
    check_cuda(cudaGetLastError(), "kernel_matmul_shmem launch");
}

// Асинхронный запуск kernel_matmul_wmma; синхронизацию выполняет вызывающая сторона.
template <unsigned int BlockItemsY = kWmmaBlockItemsY, unsigned int BlockItemsX =
     kWmmaBlockItemsX,
          unsigned int BlockItemsK = kWmmaBlockItemsK>
void launch_matmul_wmma(MatrixView<half> a, MatrixView<half> b, MatrixView<half> c,
                        cudaStream_t stream = nullptr) {
    static_assert(BlockItemsY % (kWmmaWarpRows * kWmmaFragDim) == 0,
                  "BlockItemsY must be split evenly between warp rows into 16 x 16
     fragments");
    static_assert(BlockItemsX % (kWmmaWarpCols * kWmmaFragDim) == 0,
                  "BlockItemsX must be split evenly between warp columns into 16 x 16
     fragments");
    static_assert(BlockItemsK > 0 && BlockItemsK % kWmmaFragDim == 0,
                  "BlockItemsK must be split into 16 x 16 fragments");
    static_assert((BlockItemsY * BlockItemsK) % kWmmaBlockThreads == 0 &&
                      (BlockItemsK * BlockItemsX) % kWmmaBlockThreads == 0,
                  "Tiles must be split evenly between the threads of a block");
    static_assert(is_power_of_two(BlockItemsY * BlockItemsK / kWmmaBlockThreads) &&
                      is_power_of_two(BlockItemsK * BlockItemsX / kWmmaBlockThreads),
                  "Threads must copy packets of 4, 8 or 16 bytes");
    static_assert(sizeof(half) * (BlockItemsK * (BlockItemsY + BlockItemsX) +
                                  kWmmaWarps * kWmmaFragItems) <= 48 * 1024,
                  "Tiles exceed the shared memory limit of 48 KB per block");


    if (c.size() == 0) {
        return;  // Пустая сетка — недопустимая конфигурация запуска.
    }


    const dim3 grid = matmul_grid(c, BlockItemsY, BlockItemsX);
    kernel_matmul_wmma<BlockItemsY, BlockItemsX, BlockItemsK>
        <<<grid, kWmmaBlockThreads, 0, stream>>>(a, b, c);
    check_cuda(cudaGetLastError(), "kernel_matmul_wmma launch");
}
```

Листинг А.7 – Файл `core/include/matmul_strategy.cuh`

```
#pragma once


#include <cuda_fp16.h>
#include <cuda_runtime.h>
#include <memory>
#include <type_traits>


#include "kernel.cuh"
#include "matrix_view.cuh"


// Стратегия умножения матриц на устройстве (паттерн «Стратегия»).
// Реализация получает уже согласованные по размерам представления A (m x k), B (k x
     n)
// и C (m x n) и асинхронно ставит вычисление C = A * B в поток stream.
template <typename AtomT>
class MatmulStrategy {
public:
    using Atom = AtomT;


    virtual ~MatmulStrategy() = default;
```

```
    virtual void multiply(MatrixView<AtomT> a, MatrixView<AtomT> b, MatrixView<AtomT>
     c,
                          cudaStream_t stream = nullptr) const = 0;
};


// Наивный алгоритм: каждая нить читает строку A и столбец B из глобальной памяти.
template <typename AtomT>
class NaiveMatmul final : public MatmulStrategy<AtomT> {
public:
    void multiply(MatrixView<AtomT> a, MatrixView<AtomT> b, MatrixView<AtomT> c,
                  cudaStream_t stream = nullptr) const override {
        launch_matmul_naive(a, b, c, stream);
    }
};


// Блочный алгоритм с буферизацией тайлов TileDim x TileDim в разделяемой памяти.
template <typename AtomT, unsigned int TileDim = kMatmulTileDim>
class SharedMemoryMatmul final : public MatmulStrategy<AtomT> {
public:
    void multiply(MatrixView<AtomT> a, MatrixView<AtomT> b, MatrixView<AtomT> c,
                  cudaStream_t stream = nullptr) const override {
        launch_matmul_shmem<AtomT, TileDim>(a, b, c, stream);
    }
};


// Умножение на тензорных ядрах (WMMA): только для half, размеры тайлов задают
     геометрию блока.
template <unsigned int BlockItemsY = kWmmaBlockItemsY, unsigned int BlockItemsX =
     kWmmaBlockItemsX,
          unsigned int BlockItemsK = kWmmaBlockItemsK>
class WmmaMatmul final : public MatmulStrategy<half> {
public:
    void multiply(MatrixView<half> a, MatrixView<half> b, MatrixView<half> c,
                  cudaStream_t stream = nullptr) const override {
        launch_matmul_wmma<BlockItemsY, BlockItemsX, BlockItemsK>(a, b, c, stream);
    }
};


// Стратегия, используемая Matrix::operator* по умолчанию: тензорные ядра поддерживают
     только
// half, остальные типы умножаются блочным алгоритмом с разделяемой памятью.
template <typename AtomT>
std::shared_ptr<const MatmulStrategy<AtomT>> default_matmul_strategy() {
    if constexpr (std::is_same_v<AtomT, half>) {
        static const auto strategy = std::make_shared<const WmmaMatmul<>>();
        return strategy;
    } else {
        static const auto strategy = std::make_shared<const
     SharedMemoryMatmul<AtomT>>();
        return strategy;
    }
}
```

Листинг А.8 – Файл `core/include/matrix.cuh`

```
#pragma once


#include <cstddef>
#include <memory>
#include <stdexcept>
#include <utility>


#include "cuda_check.cuh"
#include "data.cuh"
#include "matmul_strategy.cuh"
```

```
#include "matrix_view.cuh"

// Фасад над данными на устройстве.
// Агрегация: Data разделяется между копиями Matrix через std::shared_ptr.
// Композиция: каждый экземпляр Matrix владеет собственным MatrixView.
// Алгоритм умножения задаётся объектом-стратегией MatmulStrategy (по умолчанию для
     half —
// тензорные ядра, для остальных типов — блочный алгоритм с разделяемой памятью);
// стратегия разделяется между матрицами.
template <typename AtomT>
class Matrix {
public:
    using Strategy = MatmulStrategy<AtomT>;


private:
    std::shared_ptr<Data<AtomT>> data_;
    MatrixView<AtomT> view_;
    std::shared_ptr<const Strategy> matmul_strategy_;


public:
    Matrix(std::size_t nrows, std::size_t ncols,
           std::shared_ptr<const Strategy> matmul_strategy =
     default_matmul_strategy<AtomT>())
        : data_(std::make_shared<Data<AtomT>>(nrows * ncols)),
          view_(data_->data(), nrows, ncols),
          matmul_strategy_(std::move(matmul_strategy)) {
        if (!matmul_strategy_) {
            throw std::invalid_argument("Matrix: matmul strategy must not be null");
        }
    }


    std::size_t size() const { return view_.size(); }
    std::size_t nrows() const { return view_.nrows(); }
    std::size_t ncols() const { return view_.ncols(); }


    Data<AtomT>& data() { return *data_; }
    const Data<AtomT>& data() const { return *data_; }


    MatrixView<AtomT>& view() { return view_; }
    const MatrixView<AtomT>& view() const { return view_; }


    const Strategy& matmul_strategy() const { return *matmul_strategy_; }


    void set_matmul_strategy(std::shared_ptr<const Strategy> matmul_strategy) {
        if (!matmul_strategy) {
            throw std::invalid_argument("Matrix: matmul strategy must not be null");
        }
        matmul_strategy_ = std::move(matmul_strategy);
    }


    // C = (*this) * other, где (*this) — m x k, other — k x n, C — m x n.
    // Умножение выполняет стратегия левого операнда; результат наследует её.
    Matrix operator*(const Matrix& other) const {
        if (ncols() != other.nrows()) {
            throw std::invalid_argument("Matrix dimensions mismatch: lhs.ncols !=
     rhs.nrows");
        }


        Matrix result(nrows(), other.ncols(), matmul_strategy_);
        matmul_strategy_->multiply(view_, other.view_, result.view_);
        check_cuda(cudaDeviceSynchronize(), "matmul execution");
        return result;
    }
};
```

Листинг А.9 – Файл `core/src/matrix.cu`

```
#include "matrix.cuh"


// Явные инстанцирования: проверяют компилируемость шаблонов для основных типов.
template class Data<half>;
template class Data<float>;
template class Data<double>;
template class Data<int>;


template class MatrixView<half>;
template class MatrixView<float>;
template class MatrixView<double>;
template class MatrixView<int>;


template class NaiveMatmul<half>;
template class NaiveMatmul<float>;
template class NaiveMatmul<double>;
template class NaiveMatmul<int>;


template class SharedMemoryMatmul<half>;
template class SharedMemoryMatmul<float>;
template class SharedMemoryMatmul<double>;
template class SharedMemoryMatmul<int>;


template class WmmaMatmul<>;


template class Matrix<half>;
template class Matrix<float>;
template class Matrix<double>;
template class Matrix<int>;
```

Листинг А.10 – Файл `tests/CMakeLists.txt`

```
add_executable(unit_tests
    src/test_matmul.cu
)


target_include_directories(unit_tests PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)


target_compile_definitions(unit_tests PRIVATE EIGEN_NO_CUDA)


target_link_libraries(unit_tests PRIVATE
    core
    GTest::gtest_main
    Eigen3::Eigen
)


include(GoogleTest)
gtest_discover_tests(unit_tests DISCOVERY_TIMEOUT 60)
```

Листинг А.11 – Файл `tests/include/eigen_interop.cuh`

```
#pragma once


#include <Eigen/Dense>


#include "matrix.cuh"
```

```
// Eigen::MatrixXf хранит элементы по столбцам (column-major), Matrix — по строкам
     (row-major),
// поэтому обмен данными идёт через промежуточную row-major матрицу Eigen.
template <typename AtomT>
using RowMajorMatrix = Eigen::Matrix<AtomT, Eigen::Dynamic, Eigen::Dynamic,
     Eigen::RowMajor>;


// Эталон для half: совпадает с Matrix<half> по порядку хранения и формату элемента
     (binary16).
using HalfMatrix = RowMajorMatrix<Eigen::half>;


static_assert(sizeof(Eigen::half) == sizeof(half), "half types must share the binary16
     layout");


template <typename AtomT>
Matrix<AtomT> to_device(const Eigen::Matrix<AtomT, Eigen::Dynamic, Eigen::Dynamic>&
     host) {
    const RowMajorMatrix<AtomT> row_major = host;
    Matrix<AtomT> device(static_cast<std::size_t>(host.rows()),
                         static_cast<std::size_t>(host.cols()));
    device.data().copy_from_host(row_major.data());
    return device;
}


template <typename AtomT>
Eigen::Matrix<AtomT, Eigen::Dynamic, Eigen::Dynamic> to_host(const Matrix<AtomT>&
     device) {
    RowMajorMatrix<AtomT> row_major(static_cast<Eigen::Index>(device.nrows()),
                                    static_cast<Eigen::Index>(device.ncols()));
    device.data().copy_to_host(row_major.data());
    return row_major;
}


inline Matrix<half> to_device(const HalfMatrix& host) {
    Matrix<half> device(static_cast<std::size_t>(host.rows()),
                        static_cast<std::size_t>(host.cols()));
    device.data().copy_from_host(reinterpret_cast<const half*>(host.data()));
    return device;
}


inline HalfMatrix to_host(const Matrix<half>& device) {
    HalfMatrix host(static_cast<Eigen::Index>(device.nrows()),
                    static_cast<Eigen::Index>(device.ncols()));
    device.data().copy_to_host(reinterpret_cast<half*>(host.data()));
    return host;
}
```

Листинг А.12 – Файл `tests/src/test_matmul.cu`

```
#include <gtest/gtest.h>


#include <cmath>
#include <cstddef>
#include <cstdio>
#include <cstdlib>
#include <memory>
#include <string>
#include <tuple>
#include <type_traits>
#include <utility>
#include <vector>


#include "eigen_interop.cuh"
```

```
#include "matrix.cuh"

namespace {

constexpr double kPrecision = 1e-2;

const std::vector<std::size_t> kSizes = {16, 32, 64, 128, 256, 512};

// Проверяемые стратегии operator* для half.
enum class StrategyKind { Default, Naive, Shmem16, WmmaNarrow, WmmaWide, WmmaHuge };

std::shared_ptr<const MatmulStrategy<half>> make_strategy(StrategyKind kind) {
    switch (kind) {
        case StrategyKind::Naive:
            return std::make_shared<const NaiveMatmul<half>>();
        case StrategyKind::Shmem16:
            return std::make_shared<const SharedMemoryMatmul<half, 16>>();
        case StrategyKind::WmmaNarrow:
            return std::make_shared<const WmmaMatmul<32, 64, 16>>();
        case StrategyKind::WmmaWide:
            return std::make_shared<const WmmaMatmul<64, 128, 64>>();
        case StrategyKind::WmmaHuge:
            return std::make_shared<const WmmaMatmul<256, 256, 32>>();
        case StrategyKind::Default:
        default:
            return default_matmul_strategy<half>();
    }
}

std::string to_string(StrategyKind kind) {
    switch (kind) {
        case StrategyKind::Naive:
            return "Naive";
        case StrategyKind::Shmem16:
            return "Shmem16";
        case StrategyKind::WmmaNarrow:
            return "WmmaNarrow";
        case StrategyKind::WmmaWide:
            return "WmmaWide";
        case StrategyKind::WmmaHuge:
            return "WmmaHuge";
        case StrategyKind::Default:
        default:
            return "Default";
    }
}

// Случайная матрица half с элементами из [-1, 1].
HalfMatrix random_half(std::size_t rows, std::size_t cols) {
    return Eigen::MatrixXf::Random(static_cast<Eigen::Index>(rows),
     static_cast<Eigen::Index>(cols))
        .cast<Eigen::half>();
}

// Сравнение с эталоном методом Eigen::MatrixXf::isApprox с точностью kPrecision.
::testing::AssertionResult is_approx(const HalfMatrix& actual, const HalfMatrix&
     expected) {
    const Eigen::MatrixXf actual_f = actual.cast<float>();
    const Eigen::MatrixXf expected_f = expected.cast<float>();
    if (actual_f.isApprox(expected_f, kPrecision)) {
        return ::testing::AssertionSuccess();
    }
    return ::testing::AssertionFailure()
           << "max |diff| = " << (actual_f - expected_f).cwiseAbs().maxCoeff();
}
```

```
// isApprox сравнивает разность с нормой результата. При сокращении слагаемых
     результат малых
// матриц близок к нулю и этот критерий определяется округлением half, поэтому для
     вырожденных
// размеров дополнительно допускается абсолютное отклонение kPrecision по норме.
::testing::AssertionResult is_close(const HalfMatrix& actual, const HalfMatrix&
     expected) {
    const Eigen::MatrixXf difference = actual.cast<float>() - expected.cast<float>();
    if (difference.norm() <= kPrecision) {
        return ::testing::AssertionSuccess();
    }
    return is_approx(actual, expected);
}


// Точное произведение матриц half: разрядности double достаточно для сумм без
     округления.
Eigen::MatrixXd exact_product(const HalfMatrix& a, const HalfMatrix& b) {
    return a.cast<float>().cast<double>() * b.cast<float>().cast<double>();
}


double error_norm(const HalfMatrix& actual, const Eigen::MatrixXd& exact) {
    return (actual.cast<float>().cast<double>() - exact).norm();
}


}  // namespace


// ----------------------------------------------------------------------------
// Параметризованные тесты: A (m x k) * B (k x n) для всех m, n, k из kSizes.
// ----------------------------------------------------------------------------


using MatmulParams = std::tuple<std::size_t, std::size_t, std::size_t>;  // m, k, n


class MatmulParamTest : public ::testing::TestWithParam<MatmulParams> {};


// Основной набор: operator* со стратегией по умолчанию (kernel_matmul_wmma).
TEST_P(MatmulParamTest, MatchesEigen) {
    const auto [m, k, n] = GetParam();


    std::srand(static_cast<unsigned int>(m * 1'000'003 + k * 1'009 + n));
    const HalfMatrix a_host = random_half(m, k);
    const HalfMatrix b_host = random_half(k, n);
    const HalfMatrix expected = a_host * b_host;


    const Matrix<half> a = to_device(a_host);
    const Matrix<half> b = to_device(b_host);
    const Matrix<half> c = a * b;


    ASSERT_EQ(c.nrows(), m);
    ASSERT_EQ(c.ncols(), n);
    ASSERT_EQ(c.size(), m * n);


    EXPECT_TRUE(is_approx(to_host(c), expected));
}


std::string matmul_param_name(const ::testing::TestParamInfo<MatmulParams>& info) {
    const auto [m, k, n] = info.param;
    return "m" + std::to_string(m) + "_k" + std::to_string(k) + "_n" +
     std::to_string(n);
}


INSTANTIATE_TEST_SUITE_P(AllSizes, MatmulParamTest,
```

```
                         ::testing::Combine(::testing::ValuesIn(kSizes),
                                            ::testing::ValuesIn(kSizes),
                                            ::testing::ValuesIn(kSizes)),
                         matmul_param_name);

// ----------------------------------------------------------------------------
// Те же размеры для каждой стратегии в отдельности (предыдущие версии operator* и
// другие геометрии блока kernel_matmul_wmma).
// ----------------------------------------------------------------------------

using StrategyParams = std::tuple<StrategyKind, std::size_t, std::size_t,
     std::size_t>;

class MatmulStrategyParamTest : public ::testing::TestWithParam<StrategyParams> {};

TEST_P(MatmulStrategyParamTest, MatchesEigen) {
    const auto [kind, m, k, n] = GetParam();

    std::srand(static_cast<unsigned int>(m * 1'000'003 + k * 1'009 + n));
    const HalfMatrix a_host = random_half(m, k);
    const HalfMatrix b_host = random_half(k, n);
    const HalfMatrix expected = a_host * b_host;

    Matrix<half> a = to_device(a_host);
    const Matrix<half> b = to_device(b_host);
    a.set_matmul_strategy(make_strategy(kind));
    const Matrix<half> c = a * b;

    ASSERT_EQ(c.nrows(), m);
    ASSERT_EQ(c.ncols(), n);

    EXPECT_TRUE(is_approx(to_host(c), expected));
}

std::string strategy_param_name(const ::testing::TestParamInfo<StrategyParams>& info)
     {
    const auto [kind, m, k, n] = info.param;
    return to_string(kind) + "_m" + std::to_string(m) + "_k" + std::to_string(k) +
     "_n" +
          std::to_string(n);
}

INSTANTIATE_TEST_SUITE_P(AllStrategies, MatmulStrategyParamTest,
                         ::testing::Combine(::testing::Values(StrategyKind::Naive,
                                                              StrategyKind::Shmem16,

     StrategyKind::WmmaNarrow,
                                                              StrategyKind::WmmaWide,
                                                              StrategyKind::WmmaHuge),
                                            ::testing::ValuesIn(kSizes),
                                            ::testing::ValuesIn(kSizes),
                                            ::testing::ValuesIn(kSizes)),
                         strategy_param_name);

// ----------------------------------------------------------------------------
// Размеры, не кратные фрагменту и тайлам: неполные тайлы дополняются нулями, а строка
     C
// может быть не выровнена на 16 байт (ncols не кратно 8).
// ----------------------------------------------------------------------------

using UnalignedParams = std::tuple<StrategyKind, std::size_t, std::size_t,
     std::size_t>;
```

```
class MatmulUnalignedParamTest : public ::testing::TestWithParam<UnalignedParams> {};

TEST_P(MatmulUnalignedParamTest, MatchesEigen) {
    const auto [kind, m, k, n] = GetParam();

    std::srand(static_cast<unsigned int>(m * 1'000'003 + k * 1'009 + n));
    const HalfMatrix a_host = random_half(m, k);
    const HalfMatrix b_host = random_half(k, n);

    Matrix<half> a = to_device(a_host);
    const Matrix<half> b = to_device(b_host);
    a.set_matmul_strategy(make_strategy(kind));

    EXPECT_TRUE(is_close(to_host(a * b), a_host * b_host));
}

INSTANTIATE_TEST_SUITE_P(BoundarySizes, MatmulUnalignedParamTest,
                         ::testing::Combine(::testing::Values(StrategyKind::Default,
     StrategyKind::WmmaNarrow,
                                                              StrategyKind::WmmaWide,
                                                              StrategyKind::WmmaHuge),
                                            ::testing::Values(1, 15, 17, 31, 33, 63,
     129),
                                            ::testing::Values(1, 15, 17, 33, 129),
                                            ::testing::Values(1, 15, 17, 31, 33, 63,
     129)),
                         strategy_param_name);

// ---------------------------------------------------------------------------
// Граничные случаи.
// ---------------------------------------------------------------------------

TEST(MatmulTest, KnownSmallProduct) {
    // [1 2 3]   [ 7  8]   [ 58  64]
    // [4 5 6] * [ 9 10] = [139 154]
    //           [11 12]
    HalfMatrix a_host(2, 3);
    HalfMatrix b_host(3, 2);
    HalfMatrix expected(2, 2);
    a_host << Eigen::half(1), Eigen::half(2), Eigen::half(3), Eigen::half(4),
     Eigen::half(5),
        Eigen::half(6);
    b_host << Eigen::half(7), Eigen::half(8), Eigen::half(9), Eigen::half(10),
     Eigen::half(11),
        Eigen::half(12);
    expected << Eigen::half(58), Eigen::half(64), Eigen::half(139), Eigen::half(154);

    EXPECT_EQ(to_host(to_device(a_host) * to_device(b_host)), expected);
}

TEST(MatmulTest, IdentityIsNeutral) {
    constexpr std::size_t n = 129;
    std::srand(7);
    const HalfMatrix a_host = random_half(n, n);
    const HalfMatrix identity_host = HalfMatrix::Identity(n, n);

    const Matrix<half> a = to_device(a_host);
    const Matrix<half> identity = to_device(identity_host);

    EXPECT_TRUE(is_approx(to_host(a * identity), a_host));
    EXPECT_TRUE(is_approx(to_host(identity * a), a_host));
```

```
}

// Вектор-строка на вектор-столбец и наоборот: сетка из одного блока и сетка
// с вырожденной по одной оси формой; внутренняя размерность пересекает много тайлов.
TEST(MatmulTest, VectorShapes) {
    std::srand(17);
    const HalfMatrix row = random_half(1, 1000);
    const HalfMatrix col = random_half(1000, 1);

    EXPECT_TRUE(is_approx(to_host(to_device(row) * to_device(col)), row * col));
    EXPECT_TRUE(is_approx(to_host(to_device(col) * to_device(row)), col * row));
}

// Результат не зависит от количества запусков: повторное умножение не должно
// использовать данные, оставшиеся в разделяемой памяти от предыдущих запусков.
TEST(MatmulTest, RepeatedProductIsStable) {
    std::srand(19);
    const Matrix<half> a = to_device(random_half(100, 70));
    const Matrix<half> b = to_device(random_half(70, 90));

    const HalfMatrix first = to_host(a * b);
    const HalfMatrix second = to_host(a * b);

    EXPECT_EQ(first, second);
}

// Представление с невыровненным начальным адресом: и загрузка тайлов, и запись
    результата
// должны перейти на поэлементный путь, иначе векторные инструкции получат
    невыровненный адрес.
TEST(MatmulTest, UnalignedViews) {
    constexpr std::size_t m = 48;
    constexpr std::size_t k = 64;
    constexpr std::size_t n = 32;

    std::srand(41);
    const HalfMatrix a_host = random_half(m, k);
    const HalfMatrix b_host = random_half(k, n);

    Data<half> storage_a(m * k + 1);
    Data<half> storage_b(k * n + 1);
    Data<half> storage_c(m * n + 1);
    check_cuda(cudaMemcpy(storage_a.data() + 1, a_host.data(), m * k * sizeof(half),
                          cudaMemcpyHostToDevice),
               "cudaMemcpy (H2D)");
    check_cuda(cudaMemcpy(storage_b.data() + 1, b_host.data(), k * n * sizeof(half),
                          cudaMemcpyHostToDevice),
               "cudaMemcpy (H2D)");

    default_matmul_strategy<half>()->multiply(MatrixView<half>(storage_a.data() + 1,
     m, k),
                                              MatrixView<half>(storage_b.data() + 1,
     k, n),
                                              MatrixView<half>(storage_c.data() + 1,
     m, n));
    check_cuda(cudaDeviceSynchronize(), "matmul execution");

    HalfMatrix actual(m, n);
    check_cuda(cudaMemcpy(actual.data(), storage_c.data() + 1, m * n * sizeof(half),
                          cudaMemcpyDeviceToHost),
               "cudaMemcpy (D2H)");
    EXPECT_TRUE(is_approx(actual, a_host * b_host));
}
```

```
// Дополнение нулями не должно приводить к записи за границу C: результат занимает
     начало
// буфера, остаток которого играет роль охранной зоны.
TEST(MatmulTest, DoesNotWriteOutsideResult) {
    constexpr std::size_t m = 33;
    constexpr std::size_t n = 17;
    constexpr std::size_t kGuardSize = 4096;


    std::srand(29);
    const Matrix<half> a = to_device(random_half(m, 65));
    const Matrix<half> b = to_device(random_half(65, n));


    const std::vector<Eigen::half> filled(m * n + kGuardSize, Eigen::half(1.5f));
    Data<half> storage(filled.size());
    storage.copy_from_host(reinterpret_cast<const half*>(filled.data()));


    a.matmul_strategy().multiply(a.view(), b.view(), MatrixView<half>(storage.data(),
     m, n));
    check_cuda(cudaDeviceSynchronize(), "matmul execution");


    std::vector<Eigen::half> actual(filled.size());
    storage.copy_to_host(reinterpret_cast<half*>(actual.data()));
    const std::vector<Eigen::half> guard(actual.begin() + m * n, actual.end());
    EXPECT_EQ(guard, std::vector<Eigen::half>(kGuardSize, Eigen::half(1.5f)));
}


TEST(MatmulTest, OperandsAreNotModified) {
    std::srand(11);
    const HalfMatrix a_host = random_half(64, 32);
    const HalfMatrix b_host = random_half(32, 48);
    const Matrix<half> a = to_device(a_host);
    const Matrix<half> b = to_device(b_host);


    const Matrix<half> c = a * b;


    EXPECT_EQ(to_host(a), a_host);
    EXPECT_EQ(to_host(b), b_host);
    EXPECT_NE(c.data().data(), a.data().data());
    EXPECT_NE(c.data().data(), b.data().data());
}


TEST(MatmulTest, DimensionMismatchThrows) {
    const Matrix<half> a(3, 4);
    const Matrix<half> b(5, 3);
    EXPECT_THROW(a * b, std::invalid_argument);
}


TEST(MatmulTest, ZeroInnerDimensionGivesZeroMatrix) {
    const Matrix<half> a(4, 0);
    const Matrix<half> b(0, 5);


    const Matrix<half> c = a * b;


    ASSERT_EQ(c.nrows(), 4u);
    ASSERT_EQ(c.ncols(), 5u);
    EXPECT_TRUE(to_host(c).cast<float>().isZero());
}


TEST(MatmulTest, EmptyResult) {
    const Matrix<half> a(0, 3);
    const Matrix<half> b(3, 2);
```

```
    const Matrix<half> c = a * b;

    EXPECT_EQ(c.nrows(), 0u);
    EXPECT_EQ(c.ncols(), 2u);
    EXPECT_EQ(c.size(), 0u);
}

// Для типов, которые не поддерживаются тензорными ядрами, operator* работает как
     раньше.
TEST(MatmulTest, FloatPrecision) {
    std::srand(31);
    const Eigen::MatrixXf a_host = Eigen::MatrixXf::Random(129, 127);
    const Eigen::MatrixXf b_host = Eigen::MatrixXf::Random(127, 65);

    const Matrix<float> c = to_device(a_host) * to_device(b_host);

    EXPECT_TRUE(to_host(c).isApprox(a_host * b_host, 1e-5));
}

TEST(MatmulTest, DoublePrecision) {
    std::srand(13);
    const Eigen::MatrixXd a_host = Eigen::MatrixXd::Random(129, 127);
    const Eigen::MatrixXd b_host = Eigen::MatrixXd::Random(127, 3);

    const Matrix<double> c = to_device(a_host) * to_device(b_host);

    EXPECT_TRUE(to_host(c).isApprox(a_host * b_host, 1e-12));
}

TEST(MatmulTest, IntegerProductIsExact) {
    Eigen::Matrix<int, Eigen::Dynamic, Eigen::Dynamic> a_host(3, 2);
    Eigen::Matrix<int, Eigen::Dynamic, Eigen::Dynamic> b_host(2, 4);
    a_host << 1, -2, 3, 4, -5, 6;
    b_host << 1, 2, 3, 4, 5, 6, 7, 8;

    const Matrix<int> c = to_device(a_host) * to_device(b_host);

    EXPECT_EQ(to_host(c), (a_host * b_host).eval());
}

// Многократное умножение не должно приводить к утечке памяти устройства:
// после уничтожения всех матриц объём свободной памяти возвращается к исходному.
TEST(MatmulTest, NoDeviceMemoryLeak) {
    constexpr std::size_t n = 512;
    constexpr int kIterations = 50;

    // Первое обращение к CUDA и умножение инициализируют контекст и модуль заранее.
    { const Matrix<half> warm_up = Matrix<half>(n, n) * Matrix<half>(n, n); }

    std::size_t free_before = 0;
    std::size_t total = 0;
    check_cuda(cudaMemGetInfo(&free_before, &total), "cudaMemGetInfo");

    {
        const Matrix<half> a(n, n);
        const Matrix<half> b(n, n);
        for (int i = 0; i < kIterations; ++i) {
            const Matrix<half> c = a * b;
            const Matrix<half> copy = c;  // разделяет данные, не выделяет память
            Data<half> deep_copy(c.data());
        }
    }
```

```
    std::size_t free_after = 0;
    check_cuda(cudaMemGetInfo(&free_after, &total), "cudaMemGetInfo");
    EXPECT_EQ(free_after, free_before);
}

// ----------------------------------------------------------------------------
// Точность: тензорные ядра округляют сумму реже, чем последовательное накопление в
     half.
// ----------------------------------------------------------------------------

TEST(MatmulAccuracyTest, WmmaIsMoreAccurateThanSequentialHalf) {
    std::srand(37);
    const HalfMatrix a_host = random_half(256, 256);
    const HalfMatrix b_host = random_half(256, 256);
    const Eigen::MatrixXd exact = exact_product(a_host, b_host);

    const HalfMatrix wmma = to_host(to_device(a_host) * to_device(b_host));
    const HalfMatrix sequential = a_host * b_host;

    EXPECT_LT(error_norm(wmma, exact), error_norm(sequential, exact));
}

// Таблица погрешностей для отчёта (относительно точного произведения в double):
// unit_tests --gtest_also_run_disabled_tests --gtest_filter=*AccuracyReport*
TEST(MatmulAccuracyTest, DISABLED_AccuracyReport) {
    std::printf("n,method,max_abs,rms,relative\n");
    for (const std::size_t n : {16, 64, 256, 512, 1024}) {
        std::srand(static_cast<unsigned int>(n));
        const HalfMatrix a_host = random_half(n, n);
        const HalfMatrix b_host = random_half(n, n);
        const Eigen::MatrixXd exact = exact_product(a_host, b_host);

        Matrix<half> shmem_lhs = to_device(a_host);
        shmem_lhs.set_matmul_strategy(make_strategy(StrategyKind::Shmem16));
        const Matrix<half> b = to_device(b_host);

        const std::vector<std::pair<std::string, HalfMatrix>> results = {
            {"eigen_half", a_host * b_host},
            {"shmem", to_host(shmem_lhs * b)},
            {"wmma", to_host(to_device(a_host) * b)},
        };
        for (const auto& [method, result] : results) {
            const Eigen::MatrixXd error = result.cast<float>().cast<double>() - exact;
            std::printf("%zu,%s,%.5f,%.5f,%.5f\n", n, method.c_str(),
     error.cwiseAbs().maxCoeff(),
                        std::sqrt(error.array().square().mean()), error.norm() /
     exact.norm());
        }
    }
}

// ----------------------------------------------------------------------------
// Стратегии умножения.
// ----------------------------------------------------------------------------

TEST(MatmulStrategyTest, DefaultForHalfIsWmma) {
    const Matrix<half> a(2, 2);
    EXPECT_NE(dynamic_cast<const WmmaMatmul<>*>(&a.matmul_strategy()), nullptr);
}

TEST(MatmulStrategyTest, DefaultForFloatIsSharedMemory) {
    const Matrix<float> a(2, 2);
    EXPECT_NE(dynamic_cast<const SharedMemoryMatmul<float>*>(&a.matmul_strategy()),
```

```
     nullptr);
}

TEST(MatmulStrategyTest, ResultInheritsLhsStrategy) {
    const auto naive = std::make_shared<const NaiveMatmul<half>>();
    const Matrix<half> a(3, 4, naive);
    const Matrix<half> b(4, 5);

    const Matrix<half> c = a * b;

    EXPECT_EQ(&c.matmul_strategy(), naive.get());
}

TEST(MatmulStrategyTest, NullStrategyThrows) {
    EXPECT_THROW(Matrix<half>(2, 2, nullptr), std::invalid_argument);

    Matrix<half> a(2, 2);
    EXPECT_THROW(a.set_matmul_strategy(nullptr), std::invalid_argument);
}

TEST(MatmulStrategyTest, StrategiesAgree) {
    std::srand(23);
    const HalfMatrix a_host = random_half(300, 200);
    const HalfMatrix b_host = random_half(200, 100);
    Matrix<half> a = to_device(a_host);
    const Matrix<half> b = to_device(b_host);

    a.set_matmul_strategy(make_strategy(StrategyKind::Shmem16));
    const HalfMatrix shmem = to_host(a * b);
    a.set_matmul_strategy(make_strategy(StrategyKind::Default));
    const HalfMatrix wmma = to_host(a * b);

    EXPECT_TRUE(is_approx(wmma, shmem));
}

// ---------------------------------------------------------------------------
// Matrix / MatrixView / Data.
// ---------------------------------------------------------------------------

TEST(MatrixViewTest, IsTriviallyCopyable) {
    static_assert(std::is_trivially_copyable_v<MatrixView<half>>);
    static_assert(std::is_trivially_copyable_v<MatrixView<float>>);
    static_assert(std::is_trivially_copyable_v<MatrixView<double>>);
    static_assert(std::is_trivially_copyable_v<MatrixView<int>>);
}

TEST(MatrixViewTest, RowMajorIndexing) {
    std::vector<int> storage(12);
    MatrixView<int> view(storage.data(), 3, 4);
    for (std::size_t i = 0; i < view.nrows(); ++i) {
        for (std::size_t j = 0; j < view.ncols(); ++j) {
            view(i, j) = static_cast<int>(i * 10 + j);
        }
    }

    EXPECT_EQ(view.size(), 12u);
    EXPECT_EQ(view[0], 0);
    EXPECT_EQ(view[3], 3);
    EXPECT_EQ(view[4], 10);
    EXPECT_EQ(view[11], 23);
}
```

```
TEST(MatrixTest, CopiesShareData) {
    Matrix<float> a(2, 2);
    const Matrix<float> b = a;  // NOLINT: проверяем именно копирование


    EXPECT_EQ(a.data().data(), b.data().data());
    EXPECT_NE(&a.view(), &b.view());  // собственное представление у каждого
     экземпляра
    EXPECT_EQ(&a.matmul_strategy(), &b.matmul_strategy());


    const std::vector<float> values = {1, 2, 3, 4};
    a.data().copy_from_host(values.data());
    std::vector<float> read_back(4);
    b.data().copy_to_host(read_back.data());
    EXPECT_EQ(read_back, values);
}


TEST(MatrixTest, DimensionsAndSize) {
    const Matrix<half> a(3, 7);
    EXPECT_EQ(a.nrows(), 3u);
    EXPECT_EQ(a.ncols(), 7u);
    EXPECT_EQ(a.size(), 21u);
    EXPECT_EQ(a.data().size(), 21u);
}


TEST(DataTest, CopyIsDeepAndMoveTransfersOwnership) {
    const std::vector<float> values = {1, 2, 3};
    Data<float> original(3);
    original.copy_from_host(values.data());


    Data<float> copy(original);
    EXPECT_NE(copy.data(), original.data());
    std::vector<float> read_back(3);
    copy.copy_to_host(read_back.data());
    EXPECT_EQ(read_back, values);


    const float* raw = copy.data();
    Data<float> moved(std::move(copy));
    EXPECT_EQ(moved.data(), raw);
    EXPECT_EQ(copy.data(), nullptr);  // NOLINT: проверяем состояние после перемещения
    EXPECT_EQ(copy.size(), 0u);
}
```

Листинг А.13 – Файл `benchmarks/CMakeLists.txt`

```
add_executable(benchmarks
    src/bench_matmul.cu
)


target_include_directories(benchmarks PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)


target_compile_definitions(benchmarks PRIVATE EIGEN_NO_CUDA)


target_link_libraries(benchmarks PRIVATE
    core
    benchmark::benchmark
    Eigen3::Eigen
)
```

Листинг А.14 – Файл `benchmarks/include/cuda_event_timer.cuh`

```
#pragma once
```

```
#include <cuda_runtime.h>

#include "cuda_check.cuh"

// RAII-обёртка над парой CUDA Events для измерения времени работы кода на устройстве.
class CudaEventTimer {
private:
    cudaEvent_t start_{};
    cudaEvent_t stop_{};

public:
    CudaEventTimer() {
        check_cuda(cudaEventCreate(&start_), "cudaEventCreate");
        const cudaError_t err = cudaEventCreate(&stop_);
        if (err != cudaSuccess) {
            cudaEventDestroy(start_);
            check_cuda(err, "cudaEventCreate");
        }
    }

    ~CudaEventTimer() {
        cudaEventDestroy(start_);
        cudaEventDestroy(stop_);
    }

    CudaEventTimer(const CudaEventTimer&) = delete;
    CudaEventTimer& operator=(const CudaEventTimer&) = delete;

    void start(cudaStream_t stream = nullptr) {
        check_cuda(cudaEventRecord(start_, stream), "cudaEventRecord(start)");
    }

    // Возвращает время между start() и stop() в миллисекундах.
    float stop(cudaStream_t stream = nullptr) {
        check_cuda(cudaEventRecord(stop_, stream), "cudaEventRecord(stop)");
        check_cuda(cudaEventSynchronize(stop_), "cudaEventSynchronize");
        float elapsed_ms = 0.0f;
        check_cuda(cudaEventElapsedTime(&elapsed_ms, start_, stop_),
     "cudaEventElapsedTime");
        return elapsed_ms;
    }
};
```

Листинг А.15 – Файл `benchmarks/src/bench_matmul.cu`

```
#include <benchmark/benchmark.h>

#include <Eigen/Dense>
#include <algorithm>
#include <cstddef>
#include <cstdint>
#include <cstdlib>
#include <string>
#include <vector>

#include "cuda_event_timer.cuh"
#include "matrix.cuh"

namespace {

constexpr std::int64_t kMinSize = 16;
```

```
constexpr std::int64_t kMaxSize = 1024;
constexpr int kRepetitions = 10;


// Заполняет матрицу на устройстве случайными значениями из [-1, 1].
template <typename AtomT>
void fill_random(Matrix<AtomT>& matrix) {
    using RowMajorXf = Eigen::Matrix<float, Eigen::Dynamic, Eigen::Dynamic,
     Eigen::RowMajor>;
    const RowMajorXf host =
     RowMajorXf::Random(static_cast<Eigen::Index>(matrix.nrows()),

     static_cast<Eigen::Index>(matrix.ncols()));
    std::vector<AtomT> atoms(static_cast<std::size_t>(host.size()));
    std::transform(host.data(), host.data() + host.size(), atoms.begin(),
                   [](float value) { return static_cast<AtomT>(value); });
    matrix.data().copy_from_host(atoms.data());
}


// Параметры устройства попадают в поле context JSON-отчёта.
void add_device_context() {
    int device = 0;
    check_cuda(cudaGetDevice(&device), "cudaGetDevice");
    cudaDeviceProp prop{};
    check_cuda(cudaGetDeviceProperties(&prop, device), "cudaGetDeviceProperties");


    int clock_khz = 0;
    int memory_clock_khz = 0;
    check_cuda(cudaDeviceGetAttribute(&clock_khz, cudaDevAttrClockRate, device),
               "cudaDeviceGetAttribute(ClockRate)");
    check_cuda(cudaDeviceGetAttribute(&memory_clock_khz, cudaDevAttrMemoryClockRate,
     device),
               "cudaDeviceGetAttribute(MemoryClockRate)");


    benchmark::AddCustomContext("gpu_name", prop.name);
    benchmark::AddCustomContext("gpu_compute_capability",
                                std::to_string(prop.major) + "." +
     std::to_string(prop.minor));
    benchmark::AddCustomContext("gpu_sm_count",
     std::to_string(prop.multiProcessorCount));
    benchmark::AddCustomContext("gpu_clock_mhz", std::to_string(clock_khz / 1000));
    benchmark::AddCustomContext("gpu_memory_clock_mhz",
     std::to_string(memory_clock_khz / 1000));
    benchmark::AddCustomContext("gpu_memory_bus_width_bit",
                                std::to_string(prop.memoryBusWidth));
    benchmark::AddCustomContext("gpu_l2_cache_bytes",
     std::to_string(prop.l2CacheSize));
    benchmark::AddCustomContext("gpu_shared_mem_per_block_bytes",
                                std::to_string(prop.sharedMemPerBlock));
}

}  // namespace


// Matrix::operator* = выделение памяти под результат + strategy.multiply +
     синхронизация.
// Выделение/копирование/освобождение памяти вынесены за пределы цикла, а внутри цикла
// CUDA Events измеряют только время выполнения кёрнела выбранной стратегии на
     устройстве.
template <typename Strategy>
static void BM_Matrix_operator_mul(benchmark::State& state) {
    using Atom = typename Strategy::Atom;
    const auto n = static_cast<std::size_t>(state.range(0));


    std::srand(42);
    Matrix<Atom> a(n, n);
    Matrix<Atom> b(n, n);
    Matrix<Atom> c(n, n);
```

```
    fill_random(a);
    fill_random(b);


    const Strategy strategy{};
    CudaEventTimer timer;


    // Прогрев: загрузка модуля, выход GPU из режима энергосбережения.
    strategy.multiply(a.view(), b.view(), c.view());
    check_cuda(cudaDeviceSynchronize(), "warm-up");


    for (auto _ : state) {
        timer.start();
        strategy.multiply(a.view(), b.view(), c.view());
        const float elapsed_ms = timer.stop();
        state.SetIterationTime(static_cast<double>(elapsed_ms) * 1e-3);
    }


    state.counters["n"] = static_cast<double>(n);
    state.counters["flop"] = 2.0 * static_cast<double>(n) * n * n;
}

#define MATMUL_BENCHMARK(strategy)                                          \
    BENCHMARK_TEMPLATE(BM_Matrix_operator_mul, strategy)                    \
        ->RangeMultiplier(2)                                                \
        ->Range(kMinSize, kMaxSize)                                         \
        ->UseManualTime()                                                   \
        ->Repetitions(kRepetitions)                                         \
        ->ReportAggregatesOnly(true)                                        \
        ->Unit(benchmark::kMicrosecond)


// Предыдущая версия operator* (практическая работа №3): на float и на тех же данных
     half.
using ShmemFloat = SharedMemoryMatmul<float, 16>;
using ShmemHalf = SharedMemoryMatmul<half, 16>;
// Новая версия operator*.
using Wmma = WmmaMatmul<>;
// Другие геометрии блока: BlockItemsY, BlockItemsX, BlockItemsK.
using Wmma_32_64_16 = WmmaMatmul<32, 64, 16>;
using Wmma_32_64_32 = WmmaMatmul<32, 64, 32>;
using Wmma_64_64_32 = WmmaMatmul<64, 64, 32>;
using Wmma_64_128_32 = WmmaMatmul<64, 128, 32>;
using Wmma_128_128_16 = WmmaMatmul<128, 128, 16>;
using Wmma_128_128_64 = WmmaMatmul<128, 128, 64>;
using Wmma_128_256_32 = WmmaMatmul<128, 256, 32>;
using Wmma_256_256_32 = WmmaMatmul<256, 256, 32>;


MATMUL_BENCHMARK(ShmemFloat);
MATMUL_BENCHMARK(ShmemHalf);
MATMUL_BENCHMARK(Wmma);
MATMUL_BENCHMARK(Wmma_32_64_16);
MATMUL_BENCHMARK(Wmma_32_64_32);
MATMUL_BENCHMARK(Wmma_64_64_32);
MATMUL_BENCHMARK(Wmma_64_128_32);
MATMUL_BENCHMARK(Wmma_128_128_16);
MATMUL_BENCHMARK(Wmma_128_128_64);
MATMUL_BENCHMARK(Wmma_128_256_32);
MATMUL_BENCHMARK(Wmma_256_256_32);


// Контрольные размеры сверх задания: при n <= 512 блоков kernel_matmul_wmma меньше,
     чем SM,
// поэтому кубическую асимптотику можно проверить только на больших матрицах.
#define MATMUL_LARGE_BENCHMARK(strategy)                                    \
    BENCHMARK_TEMPLATE(BM_Matrix_operator_mul, strategy)                    \
        ->Arg(2048)                                                         \
        ->Arg(4096)                                                         \
```

```
        ->UseManualTime()                                                \
        ->Repetitions(kRepetitions)                                      \
        ->ReportAggregatesOnly(true)                                     \
        ->Unit(benchmark::kMicrosecond)

MATMUL_LARGE_BENCHMARK(ShmemFloat);
MATMUL_LARGE_BENCHMARK(ShmemHalf);
MATMUL_LARGE_BENCHMARK(Wmma);
MATMUL_LARGE_BENCHMARK(Wmma_32_64_32);

// Калибровка: предельный темп операций mma_sync без обращений к памяти. Сетка из 120
     блоков
// занимает все слоты 20 SM (6 блоков по 256 нитей); накопитель записывается в sink,
     чтобы
// компилятор не удалил цикл.
__global__ void kernel_mma_peak(half* sink, int iterations) {
    namespace wmma = nvcuda::wmma;

    wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> frag_a;
    wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::row_major> frag_b;
    wmma::fragment<wmma::accumulator, 16, 16, 16, half> acc;
    wmma::fill_fragment(frag_a, __float2half(1e-3f));
    wmma::fill_fragment(frag_b, __float2half(1e-3f));
    wmma::fill_fragment(acc, half{});

    for (int i = 0; i < iterations; ++i) {
        wmma::mma_sync(acc, frag_a, frag_b, acc);
    }

    const unsigned int warp = blockIdx.x * kWmmaWarps + threadIdx.x / kWarpSize;
    wmma::store_matrix_sync(sink + warp * kWmmaFragItems, acc, kWmmaFragDim,
     wmma::mem_row_major);
}

static void BM_Wmma_peak(benchmark::State& state) {
    constexpr unsigned int kBlocks = 120;
    constexpr int kIterations = 16384;

    Data<half> sink(kBlocks * kWmmaWarps * kWmmaFragItems);
    CudaEventTimer timer;

    kernel_mma_peak<<<kBlocks, kWmmaBlockThreads>>>(sink.data(), kIterations);
    check_cuda(cudaDeviceSynchronize(), "warm-up");

    for (auto _ : state) {
        timer.start();
        kernel_mma_peak<<<kBlocks, kWmmaBlockThreads>>>(sink.data(), kIterations);
        const float elapsed_ms = timer.stop();
        check_cuda(cudaGetLastError(), "kernel_mma_peak launch");
        state.SetIterationTime(static_cast<double>(elapsed_ms) * 1e-3);
    }

    // Одна операция mma_sync — 16^3 умножений и столько же сложений.
    state.counters["flop"] = 2.0 * kBlocks * kWmmaWarps * kIterations * 4096.0;
}

BENCHMARK(BM_Wmma_peak)
    ->UseManualTime()
    ->Repetitions(kRepetitions)
    ->ReportAggregatesOnly(true)
    ->Unit(benchmark::kMicrosecond);

int main(int argc, char** argv) {
```

```
    benchmark::Initialize(&argc, argv);
    if (benchmark::ReportUnrecognizedArguments(argc, argv)) {
        return 1;
    }
    add_device_context();
    benchmark::RunSpecifiedBenchmarks();
    benchmark::Shutdown();
    return 0;
}
```

Листинг А.16 – Файл `scripts/bench_data.py`

```
"""Загрузка результатов Google Benchmark (JSON) для построения графиков."""

import json
from dataclasses import dataclass
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
DEFAULT_JSON = ROOT / "results" / "benchmark_results.json"
LARGE_JSON = ROOT / "results" / "benchmark_results_large.json"
IMAGES_DIR = ROOT / "docs" / "images"

SHMEM_FLOAT = "BM_Matrix_operator_mul<ShmemFloat>"
SHMEM_HALF = "BM_Matrix_operator_mul<ShmemHalf>"
WMMA = "BM_Matrix_operator_mul<Wmma>"
WMMA_MIN = "BM_Matrix_operator_mul<Wmma_32_64_32>"
PEAK = "BM_Wmma_peak"

LABELS = {
    SHMEM_FLOAT: "kernel_matmul_shmem, float (работа №3)",
    SHMEM_HALF: "kernel_matmul_shmem, half (работа №3)",
    WMMA: "kernel_matmul_wmma, тайл 128×128×32",
    WMMA_MIN: "kernel_matmul_wmma, тайл 32×64×32",
}

COLORS = {SHMEM_FLOAT: "#7570b3", SHMEM_HALF: "#d95f02", WMMA: "#1b9e77", WMMA_MIN:
     "#66a61e"}
SYMBOLS = {SHMEM_FLOAT: "diamond", SHMEM_HALF: "circle", WMMA: "square", WMMA_MIN:
     "triangle-up"}

# Типичная частота GPU под нагрузкой (nvidia-smi --query-gpu=clocks.gr), МГц:
# cudaDevAttrClockRate возвращает 1485 МГц, а во время измерений частота составляла
     1845–1950 МГц.
LOADED_CLOCK_MHZ = 1900

# Постоянные накладные расходы на запуск кёрнела и запись событий, с (медиана T при n
     <= 64).
OVERHEAD_SIZES = (16, 32, 64)

# Множители перевода в секунды для поля time_unit.
_TO_SECONDS = {"ns": 1e-9, "us": 1e-6, "ms": 1e-3, "s": 1.0}



@dataclass
class Series:
    sizes: list[int]
    median_s: list[float]
    stddev_s: list[float]



def load_json(json_path: Path = DEFAULT_JSON) -> dict:
```

```
    # На Windows путь к исполняемому файлу в "context" пишется в ANSI-кодировке.
    with open(json_path, encoding="utf-8", errors="replace") as f:
        return json.load(f)


def load(json_path: Path = DEFAULT_JSON, large: bool = False) -> dict[str, Series]:
    """Возвращает {имя бенчмарка: Series} по агрегатам median и stddev.

    large=True добавляет контрольные размеры сверх задания (2048 и 4096).
    """
    benchmarks = load_json(json_path)["benchmarks"]
    if large:
        benchmarks = benchmarks + load_json(LARGE_JSON)["benchmarks"]

    stats: dict[str, dict[int, dict[str, float]]] = {}
    for bench in benchmarks:
        aggregate = bench.get("aggregate_name")
        if aggregate not in ("median", "stddev"):
            continue
        parts = bench["run_name"].split("/")
        if not parts[1].isdigit():  # калибровочный бенчмарк без размера матрицы
            continue
        seconds = bench["real_time"] * _TO_SECONDS[bench["time_unit"]]
        stats.setdefault(parts[0], {}).setdefault(int(parts[1]), {})[aggregate] =
     seconds

    result = {}
    for family, by_size in stats.items():
        sizes = sorted(by_size)
        result[family] = Series(
            sizes=sizes,
            median_s=[by_size[n]["median"] for n in sizes],
            stddev_s=[by_size[n].get("stddev", 0.0) for n in sizes],
        )
    return result


def peak_gflops(json_path: Path = DEFAULT_JSON) -> float:
    """Измеренный предел тензорных ядер, GFLOP/s (калибровочный бенчмарк
     BM_Wmma_peak)."""
    for bench in load_json(json_path)["benchmarks"]:
        if bench["run_name"].startswith(PEAK) and bench.get("aggregate_name") ==
     "median":
            seconds = bench["real_time"] * _TO_SECONDS[bench["time_unit"]]
            return bench["flop"] / seconds / 1e9
    raise ValueError(f"{PEAK} not found in {json_path}")


def geometry(family: str) -> tuple[int, int, int]:
    """(BlockItemsY, BlockItemsX, BlockItemsK) для бенчмарка kernel_matmul_wmma."""
    if family == WMMA:
        return 128, 128, 32
    y, x, k = family.split("<")[1].rstrip(">").split("_")[1:]
    return int(y), int(x), int(k)


def is_wmma(family: str) -> bool:
    return "<Wmma" in family


def gflops(n: int, seconds: float) -> float:
    """Производительность умножения n x n матриц: 2 n^3 операций."""
```

```
    return 2.0 * n**3 / seconds / 1e9


def save(fig, name: str) -> None:
    """Сохраняет график в docs/images в форматах PNG и HTML (интерактивный)."""
    IMAGES_DIR.mkdir(parents=True, exist_ok=True)
    fig.write_html(IMAGES_DIR / f"{name}.html", include_plotlyjs="cdn")
    try:
        fig.write_image(IMAGES_DIR / f"{name}.png", width=1000, height=600, scale=2)
        print(f"Saved {IMAGES_DIR / name}.png / .html")
    except Exception as exc:  # kaleido требует установленный Chrome
        print(f"PNG export failed ({exc}); saved {IMAGES_DIR / name}.html only")
```

Листинг А.17 – Файл `scripts/complexity_chart.py`

```
"""График реальной вычислительной сложности T(n) для умножения n x n матриц.

Запуск из корня проекта: python scripts/complexity_chart.py
     [path/to/benchmark_results.json]
"""


import sys
from pathlib import Path

import numpy as np
import plotly.graph_objects as go

from bench_data import (COLORS, DEFAULT_JSON, LABELS, SHMEM_FLOAT, SHMEM_HALF,
     SYMBOLS, WMMA,
                        WMMA_MIN, gflops, load, save)

FAMILIES = (SHMEM_FLOAT, SHMEM_HALF, WMMA, WMMA_MIN)



def fit_exponent(sizes: list[int], times: list[float], min_size: int) -> float:
    """Показатель степени p в T(n) ~ c * n^p (МНК в логарифмических координатах)."""
    pts = [(n, t) for n, t in zip(sizes, times) if n >= min_size]
    log_n, log_t = np.log([p[0] for p in pts]), np.log([p[1] for p in pts])
    return float(np.polyfit(log_n, log_t, 1)[0])



def main() -> None:
    json_path = Path(sys.argv[1]) if len(sys.argv) > 1 else DEFAULT_JSON
    series = load(json_path)

    fig = go.Figure()
    for family in FAMILIES:
        s = series[family]
        fig.add_trace(go.Scatter(
            x=s.sizes, y=[t * 1e3 for t in s.median_s],
            error_y=dict(type="data", array=[d * 1e3 for d in s.stddev_s],
     visible=True),
            mode="lines+markers", name=LABELS[family],
            line=dict(color=COLORS[family], width=2),
            marker=dict(symbol=SYMBOLS[family], size=9),
        ))

    # Теоретическая оценка O(n^3), привязанная к последней точке измерений.
    for family, label in ((SHMEM_HALF, "shmem"), (WMMA, "wmma")):
        s = series[family]
        n_max, t_max = s.sizes[-1], s.median_s[-1]
```

```
        fig.add_trace(go.Scatter(
            x=s.sizes, y=[t_max * (n / n_max) ** 3 * 1e3 for n in s.sizes],
            mode="lines", name=f"c·n³ ({label})",
            line=dict(color=COLORS[family], width=1, dash="dash"),
        ))

    fig.update_layout(
        title="Реальная вычислительная сложность умножения матриц n×n",
        xaxis=dict(title="Размер матрицы n", type="log", tickvals=series[WMMA].sizes),
        # Диапазон ограничен измерениями: теоретические прямые при малых n ушли бы на
     порядки ниже.
        yaxis=dict(title="Время T(n), мс", type="log", exponentformat="power",
                   range=[np.log10(5e-3), np.log10(10.0)]),
        template="plotly_white", legend=dict(x=0.01, y=0.99),
    )
    save(fig, "complexity_chart")


    # Таблица для отчёта.
    header = f"{'n':>6}" + "".join(f" | {name.split('<')[1][:-1]:>14}, мкс | GFLOP/s"
                                   for name in FAMILIES)
    print("\n" + header)
    for i, n in enumerate(series[WMMA].sizes):
        row = f"{n:>6}"
        for family in FAMILIES:
            t = series[family].median_s[i]
            row += f" | {t * 1e6:>19.1f} | {gflops(n, t):>7.1f}"
        print(row)


    # Показатель степени по диапазону задания и по контрольным размерам сверх него.
    extended = load(json_path, large=True)
    for family in FAMILIES:
        s, e = series[family], extended[family]
        print(f"{family}: p(n >= 16) = {fit_exponent(s.sizes, s.median_s, 16):.2f}, "
              f"p(n >= 256) = {fit_exponent(s.sizes, s.median_s, 256):.2f}, "
              f"p(n >= 1024) = {fit_exponent(e.sizes, e.median_s, 1024):.2f}")
        print("    extended, us: " + ", ".join(f"{n}: {t * 1e6:.1f}" for n, t in
                                               zip(e.sizes, e.median_s) if n >= 1024))



if __name__ == "__main__":
    main()
```

Листинг А.18 – Файл `scripts/speedup_chart.py`

```
"""График ускорения S(n) = T_shmem(n) / T_wmma(n) новой реализации operator*.

Запуск из корня проекта: python scripts/speedup_chart.py
     [path/to/benchmark_results.json]
"""


import math
import sys
from pathlib import Path


import plotly.graph_objects as go


from bench_data import (COLORS, DEFAULT_JSON, SHMEM_FLOAT, SHMEM_HALF, SYMBOLS, WMMA,
     load, save)


BASELINES = {
    SHMEM_HALF: ("относительно kernel_matmul_shmem на half", "top left"),
    SHMEM_FLOAT: ("относительно kernel_matmul_shmem на float", "bottom right"),
}
```

```
def main() -> None:
    json_path = Path(sys.argv[1]) if len(sys.argv) > 1 else DEFAULT_JSON
    series = load(json_path)


    wmma = dict(zip(series[WMMA].sizes, series[WMMA].median_s))


    fig = go.Figure()
    for family, (label, position) in BASELINES.items():
        baseline = dict(zip(series[family].sizes, series[family].median_s))
        sizes = sorted(wmma.keys() & baseline.keys())
        speedup = [baseline[n] / wmma[n] for n in sizes]


        fig.add_trace(go.Scatter(
            x=sizes, y=speedup, mode="lines+markers+text", name=label,
            text=[f"{s:.2f}×" for s in speedup], textposition=position,
            line=dict(color=COLORS[family], width=2),
            marker=dict(symbol=SYMBOLS[family], size=9),
        ))


        print(label)
        for n, s in zip(sizes, speedup):
            print(f"  n = {n:>5}: S = {s:6.2f}")


    fig.add_hline(y=1.0, line=dict(color="red", dash="dash"))  # граница S = 1


    fig.update_layout(
        title="Ускорение kernel_matmul_wmma относительно предыдущей версии operator*",
        xaxis=dict(title="Размер матрицы n", type="log", tickvals=series[WMMA].sizes,
                   range=[math.log10(11), math.log10(1800)]),
        yaxis=dict(title="Ускорение S(n) = T_shmem / T_wmma, раз"),
        template="plotly_white", legend=dict(x=0.01, y=0.99),
    )
    save(fig, "speedup_chart")



if __name__ == "__main__":
    main()
```

Листинг А.19 – Файл `scripts/performance_chart.py`

```
"""График производительности P(n) = 2n^3 / T(n) с измеренным пределом тензорных ядер.


Предел получен калибровочным бенчмарком BM_Wmma_peak (mma_sync без обращений к
     памяти),
предел FP32 — расчётный: 128 FMA/такт на SM для kernel_matmul_shmem.


Запуск из корня проекта: python scripts/performance_chart.py
     [path/to/benchmark_results.json]
"""


import math
import sys
from pathlib import Path


import plotly.graph_objects as go


from bench_data import (COLORS, DEFAULT_JSON, LABELS, LOADED_CLOCK_MHZ, SHMEM_FLOAT,
     SHMEM_HALF,
```

```
                        SYMBOLS, WMMA, WMMA_MIN, gflops, load, load_json, peak_gflops,
    save)

FP32_LANES_PER_SM = 128



def main() -> None:
    json_path = Path(sys.argv[1]) if len(sys.argv) > 1 else DEFAULT_JSON
    series = load(json_path)
    peak = peak_gflops(json_path)


    sm_count = int(load_json(json_path)["context"]["gpu_sm_count"])
    clock_ghz = LOADED_CLOCK_MHZ / 1e3
    fp32_peak = 2.0 * FP32_LANES_PER_SM * sm_count * clock_ghz


    fig = go.Figure()
    for family in (SHMEM_FLOAT, SHMEM_HALF, WMMA, WMMA_MIN):
        s = series[family]
        fig.add_trace(go.Scatter(
            x=s.sizes, y=[gflops(n, t) for n, t in zip(s.sizes, s.median_s)],
            mode="lines+markers", name=LABELS[family],
            line=dict(color=COLORS[family], width=2),
            marker=dict(symbol=SYMBOLS[family], size=9),
        ))


    for value, family, text in ((peak, WMMA, "предел тензорных ядер (измерен)"),
                                (fp32_peak, SHMEM_FLOAT, "предел FP32 (расчёт)")):
        fig.add_hline(y=value, line=dict(color=COLORS[family], dash="dash", width=1))
        # На логарифмической оси координата подписи задаётся десятичным логарифмом.
        fig.add_annotation(xref="paper", x=0.01, y=math.log10(value), yshift=11,
     showarrow=False,
                           xanchor="left", text=f"{text}: {value:.0f} GFLOP/s")


    fig.update_layout(
        title="Производительность умножения матриц n×n",
        xaxis=dict(title="Размер матрицы n", type="log", tickvals=series[WMMA].sizes),
        yaxis=dict(title="Производительность P(n), GFLOP/s", type="log",
     exponentformat="power",
                   range=[math.log10(0.2), math.log10(peak * 2.5)]),
        template="plotly_white", legend=dict(x=0.62, y=0.05, yanchor="bottom"),
    )
    save(fig, "performance_chart")


    print(f"wmma peak: {peak:.0f} GFLOP/s = "
          f"{peak / 2 / sm_count / clock_ghz:.0f} FMA/clock/SM at {LOADED_CLOCK_MHZ}
     MHz")
    print(f"fp32 peak: {fp32_peak:.0f} GFLOP/s")



if __name__ == "__main__":
    main()
```

Листинг А.20 – Файл `scripts/geometry_chart.py`

```
"""Влияние геометрии блока kernel_matmul_wmma: модель T(n) = t0 + 2n^3/P + V/B.


t0 — накладные расходы запуска, 2n^3/P — работа тензорных ядер с измеренным пределом
     P,
V = 2 n^3 (1/X + 1/Y) байт — объём загрузки тайлов из глобальной памяти (X, Y —
     стороны тайла C).
Эффективная пропускная способность B оценивается МНК по геометриям с полной загрузкой
     SM.
```

```
Запуск из корня проекта: python scripts/geometry_chart.py
     [path/to/benchmark_results.json]
"""


import sys
from pathlib import Path


import numpy as np
import plotly.graph_objects as go


from bench_data import (COLORS, DEFAULT_JSON, OVERHEAD_SIZES, WMMA, geometry, is_wmma,
     load,
                        peak_gflops, save)


FIT_SIZE = 1024
MIN_BLOCKS = 40  # не меньше двух блоков на SM: сетка занимает все 20 SM
MAX_SHARED_KB = 128 / 6  # разделяемая память не ограничивает число блоков на SM (6
     слотов)



def main() -> None:
    json_path = Path(sys.argv[1]) if len(sys.argv) > 1 else DEFAULT_JSON
    series = load(json_path)
    peak_flops = peak_gflops(json_path) * 1e9


    overhead = float(np.median([t for s in series.values()
                                for n, t in zip(s.sizes, s.median_s) if n in
     OVERHEAD_SIZES]))


    rows = []
    for family, s in series.items():
        if not is_wmma(family):
            continue
        y, x, k = geometry(family)
        t = dict(zip(s.sizes, s.median_s))[FIT_SIZE]
        blocks = (FIT_SIZE // x) * (FIT_SIZE // y)
        shared_kb = 2.0 * (k * (x + y) + 8 * 256) / 1024  # тайлы A, B и буфер записи
     C
        rows.append(dict(family=family, y=y, x=x, k=k, t=t, blocks=blocks,
     shared_kb=shared_kb,
                         volume=2.0 * FIT_SIZE**3 * (1 / x + 1 / y)))
    rows.sort(key=lambda r: r["volume"])


    # T - t0 - 2n^3/P = V / B: прямая через начало координат.
    compute_s = 2.0 * FIT_SIZE**3 / peak_flops
    fit = [r for r in rows if r["blocks"] >= MIN_BLOCKS and r["shared_kb"] <=
     MAX_SHARED_KB]
    volume = np.array([r["volume"] for r in fit])
    load_s = np.array([r["t"] - overhead - compute_s for r in fit])
    bandwidth = float(volume @ volume / (volume @ load_s))


    names = [f"{r['y']}×{r['x']}×{r['k']}" for r in rows]
    modelled = [overhead + compute_s + r["volume"] / bandwidth for r in rows]


    fig = go.Figure()
    fig.add_trace(go.Bar(x=names, y=[r["t"] * 1e6 for r in rows], name="Измерено",
                         marker_color=[COLORS[WMMA] if r["family"] == WMMA else
     "#a6a6a6"
                                       for r in rows]))
    fig.add_trace(go.Scatter(x=names, y=[t * 1e6 for t in modelled], name="Модель t₀ +
     2n³/P + V/B",
                             mode="lines+markers", line=dict(color="#d95f02",
```

```
     width=2)))
    fig.update_layout(
        title=f"Время умножения матриц {FIT_SIZE}×{FIT_SIZE} при разной геометрии
     блока",
        xaxis=dict(title="Тайлы BlockItemsY × BlockItemsX × BlockItemsK"),
        yaxis=dict(title="Время T, мкс", range=[0, 620]),
        template="plotly_white", legend=dict(x=0.01, y=0.99),
    )
    save(fig, "geometry_chart")

    print(f"t0 = {overhead * 1e6:.1f} us, 2n^3/P = {compute_s * 1e6:.1f} us, "
          f"B = {bandwidth / 1e9:.1f} GB/s")
    print(f"{'geometry':>14} {'blocks':>7} {'shared':>7} {'V, MB':>8} {'T, us':>8}
     {'model':>8} "
          f"{'P_kernel':>9}")
    for name, r, m in zip(names, rows, modelled):
        print(f"{name:>14} {r['blocks']:>7} {r['shared_kb']:>7.1f} {r['volume'] /
     1e6:>8.1f} "
              f"{r['t'] * 1e6:>8.1f} {m * 1e6:>8.1f} {2.0 * FIT_SIZE**3 / r['t'] /
     1e9:>9.0f}")



if __name__ == "__main__":
    main()
```